
JavaScript Básico: El Poder de la Interactividad

Una Novela para Aprender JavaScript Desde Cero

Alex Goyzueta Delgado
alexgoyzueta2018@gmail.com

JAVASCRIPT BÁSICO

El Poder de la Interactividad

Una Novela para Aprender JavaScript Desde Cero

Autor: Alex Goyzueta Delgado

Contacto: alexgoyzueta2018@gmail.com

> *"La web cobra vida cuando aprendes a hablar su idioma.*

> *JavaScript no es solo un lenguaje:*

> *es la chispa que enciende la experiencia digital."*

Lima, 2026

Créditos

JavaScript Básico: El Poder de la Interactividad

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en forma alguna ni por ningún medio electrónico, mecánico, fotocopia, grabación u otro método, sin el permiso previo por escrito del autor.

Edición y corrección: Alex Goyzueta Delgado

Ilustraciones conceptuales: Generadas mediante descripciones textuales

Diseño de portada: Alex Goyzueta Delgado

Tipografía: IBM Plex Mono (código), Lato (narrativa)

Agradecimientos especiales:

A los desarrolladores que comparten su conocimiento en comunidades abiertas.

A los diseñadores que se aventuran en el código y descubren un nuevo mundo.

A todos los que creen que la interactividad transforma lo estático en experiencia.

Datos de catalogación:

Goyzueta Delgado, Alex

JavaScript Básico: El Poder de la Interactividad / Alex Goyzueta Delgado

1ra edición — Lima, 2026

Dedicatoria

A los diseñadores que se atreven a mirar detrás del lienzo
y descubren que el código también es arte.
A los que creen que una página web puede ser más que texto estático:
un diálogo entre la máquina y el ser humano.
A los que empiezan sin saber y terminan creando.
Que este libro sea tu primer ``console.log``
en una conversación que durará toda la vida.
"La interactividad no es una característica: es el alma de la web."
— Camila Ríos

Prefacio

¿Por qué este libro?

JavaScript es el lenguaje más popular de la web. Está en todos lados: en los botones que presionas, en los formularios que llenas, en las animaciones que te sorprenden, en las aplicaciones que usas a diario. Pero aprenderlo puede ser abrumador si empiezas con manuales técnicos llenos de sintaxis abstracta.

Este libro es diferente.

JavaScript Básico: El Poder de la Interactividad es una novela educativa. Acompaña a Camila Ríos, una diseñadora UX que nunca ha programado, en su misión de convertir una plataforma educativa estática en una experiencia interactiva. Ella comete tus mismos errores, hace tus mismas preguntas, y celebra tus mismos triunfos.

¿Cómo usar este libro?

Cada capítulo técnico tiene tres partes:

1. **Narrativa** — La historia de Camila y su equipo en "AprendeYa". Un problema del mundo real que necesita una solución JavaScript.
2. **Explicación técnica** — Los conceptos que Camila aprende para resolver sus problemas. Con ejemplos claros y funcionales.
3. **Enigmas** — Ejercicios para que practiques. Al final del libro encontrarás las soluciones.

¿Qué necesitas?

- Un navegador web moderno (Chrome, Firefox, Edge)
- Visual Studio Code (o cualquier editor de código)
- Curiosidad y ganas de crear

Un viaje interactivo

Al terminar este libro, no solo sabrás qué pasó con la plataforma "AprendeYa". Habrás aprendido JavaScript desde las variables hasta la manipulación del DOM, pasando por funciones, arrays, objetos, eventos, formularios, almacenamiento local y consumo de APIs.

Y descubrirás que JavaScript no es solo un lenguaje de programación. Es el poder de hacer que la web cobre vida.

— Alex Goyzueta Delgado

Lima, 2026

Introducción

Lima, 2026

El sol de la mañana se colaba por las persianas de la oficina en el piso 14. Frente a una pantalla dividida entre Figma y una pestaña del navegador, Camila Ríos ajustaba el último prototipo de la plataforma "AprendeYa". Tenía 27 años, era diseñadora UX, y amaba su trabajo.

—Está hermoso —dijo Valeria, su compañera de equipo, asomándose por encima del monitor—. Los botones, los colores, la tipografía. Pero...

—Ya sé —suspiró Camila—. No hace nada.

El problema

"AprendeYa" era una startup educativa que había desarrollado una plataforma de cursos online. La interfaz era impecable: diseño minimalista, paleta de colores armónica, tipografía legible. Todo en HTML y CSS. Navegación fluida... pero estática.

Los usuarios podían ver el contenido, pero no interactuar con él. Los botones eran decorativos. Los formularios no validaban datos. Los cursos se mostraban siempre igual, sin importar quién los viera.

Esa mañana, el CEO convocó una reunión de emergencia.

—Tres semanas —dijo Marco, el CEO, con el rostro serio—. En tres semanas tenemos la demo con inversionistas. Y necesito que la plataforma haga algo. Cualquier cosa. Registrar usuarios, mostrar su progreso, recomendar cursos. Algo que demuestre que esto es más que un prototipo bonito.

Todos los ojos miraron a Camila. Ella era la diseñadora, no la programadora. Pero era también la que mejor conocía la plataforma.

—Yo... nunca he programado —dijo Camila—. HTML y CSS, sí. Pero JavaScript...

—Te conseguimos ayuda —la interrumpió Marco—. Diego está libre desde la semana que viene.

Diego era el desarrollador backend de la empresa. Un tipo tranquilo, de gafas redondas y sudadera con capucha, que pasaba la mayor parte del tiempo en la terminal del servidor. Tenía fama de ser paciente y buen profesor.

Lo que está en juego

La startup había gastado su capital semilla en desarrollar la plataforma. Si la demo fallaba, no habría segunda ronda de inversión. Veinte empleados perderían su trabajo. Meses de trabajo se desvanecerían.

Camila respiró hondo. Recordó por qué había estudiado diseño: para crear experiencias que importaran. Y ahora tenía la oportunidad de llevar su diseño al siguiente nivel.

—Está bien —dijo—. Lo haré.

El viaje comienza

Esa tarde, Diego se acercó a su escritorio con una laptop bajo el brazo y una taza de café en la mano.

—¿Lista para darle vida a esta página? —preguntó con una sonrisa.

Camila asintió, con el corazón latiendo rápido. No sabía lo que le esperaba, pero estaba decidida a aprender.

¿Qué aprenderás en este libro?

Acompaña a Camila en su viaje. A través de diez capítulos, aprenderás:

1	Variables y tipos de datos	El ADN de todo programa JavaScript
3	Funciones	Modularizar y reutilizar lógica
5	DOM y eventos	Hacer que la página reaccione al usuario
7	Formularios y validación	Capturar datos de forma inteligente
9	Fetch y APIs	Conectarse con el mundo exterior

Cada capítulo incluye ejemplos de código y enigmas para practicar. Al final del libro encontrarás las soluciones.

La plataforma "AprendeYa" te espera. ¿Estás listo para darle vida?

Capítulo 1: El Primer Script

Introducción a JS, Variables y Tipos de Datos

—¿Por dónde empezamos? —preguntó Camila, con el cursor parpadeando en una pantalla en blanco del navegador.

Diego se sentó a su lado, abrió Chrome y presionó F12.

—El navegador tiene herramientas ocultas —dijo—. Esto se llama **Consola del Desarrollador**. Es como el taller del mecánico, pero para páginas web.

Camila vio cómo se abría un panel dividido: HTML en una pestaña, red en otra, y una pestaña llamada "Console" con un cursor parpadeante.

—Escribe esto —dijo Diego, pasándole una hoja con una línea escrita:

```
console.log("Hola, AprendeYa!");
```

Camila tecleó la línea y presionó Enter. En la consola apareció:

```
Hola, AprendeYa!
```

—¡Funcionó! —exclamó—. ¿Qué acabo de hacer?

—Acabas de escribir tu primer **script** en **JavaScript**. El lenguaje que le da vida a las páginas web.

¿Qué es JavaScript?

JavaScript es un lenguaje de programación que se ejecuta en el navegador del usuario. Mientras que **HTML** define la estructura de una página y **CSS** define su apariencia, JavaScript define su **comportamiento**.

Tecnología	Función	Analogía
HTML	Estructura	El esqueleto de una casa
CSS	Apariencia	Las paredes pintadas, los muebles
JavaScript	Comportamiento	Las tuberías, la electricidad, la calefacción

—¿Y por qué se llama JavaScript si no tiene nada que ver con Java? —preguntó Camila. Diego rió.

—Buena pregunta. Fue una estrategia de marketing en los 90. Java era muy popular, así que

llamaron a este nuevo lenguaje "JavaScript" para montarse en la ola. No tienen nada que ver.

¿Dónde se ejecuta JavaScript?

JavaScript se puede ejecutar en:

1. **El navegador** — La forma más común. Chrome, Firefox, Edge, Safari.
2. **Node.js** — Un entorno que permite ejecutar JavaScript fuera del navegador (en el servidor).
3. **Aplicaciones híbridas** — Como Electron (VS Code, Slack, Discord).

Para este libro, trabajaremos exclusivamente en el navegador.

¿Cómo incluir JavaScript en una página?

Hay tres formas:

1. JavaScript en línea (inline):

```
<button onclick="alert('Hola!')">Haz clic</button>
```

2. Dentro de una etiqueta `<script>`:

```
<script>
  console.log("Esto está dentro del HTML");
</script>
```

3. Archivo externo (recomendado):

```
<script src="script.js"></script>
```

La consola del navegador

La **consola** es la herramienta más importante para un desarrollador JavaScript. Permite:

- Ejecutar código en tiempo real
- Ver mensajes de error
- Inspeccionar el valor de las variables
- Depurar el comportamiento del programa

Para abrirla:

Navegador	Atajo Windows	Atajo Mac
Chrome	F12 o Ctrl + Shift + I	Cmd + Option + I
Firefox	F12 o Ctrl + Shift + I	Cmd + Option + I
Edge	F12 o Ctrl + Shift + I	Cmd + Option + I

console.log()

La función `console.log()` imprime un mensaje en la consola. Es la forma más básica de ver qué está pasando en tu código.

```
console.log("Mensaje de texto");
console.log(42);
console.log(true);
console.log([1, 2, 3]);
```

```
console.log({ nombre: "Camila" });
```

—Úsalo siempre que tengas dudas —dijo Diego—. Es como decirle al código "muéstrame qué tienes aquí".

Variables: guardando información

—Ahora —continuó Diego—, imagina que necesitas guardar el nombre de un usuario, su edad, o el título de un curso. Para eso sirven las **variables**.

Las variables son contenedores que almacenan datos. En JavaScript hay tres formas de declararlas:

var (la forma antigua)

```
var nombre = "Camila";  
var edad = 27;
```

`var` tiene un comportamiento especial con el **scope** que puede causar bugs difíciles de encontrar. Hoy en día se recomienda no usarlo.

let (la forma moderna)

```
let nombre = "Camila";  
let edad = 27;  
nombre = "Valeria"; // Se puede reasignar
```

`let` se usa para variables que pueden cambiar de valor.

const (constante)

```
const PI = 3.1416;  
const SITE_NAME = "AprendeYa";  
// PI = 3; // Error: no se puede reasignar una constante
```

`const` se usa para valores que no deben cambiar. Es inmutable (no se puede reasignar).

—La regla de oro —dijo Diego— es: usa `const` por defecto. Solo usa `let` cuando sepas que la variable va a cambiar. Nunca uses `var` en código moderno.

Declaración	Se puede reasignar	Scope	Recomendado
`var`	Sí	Función	No
`let`	Sí	Bloque	Sí (cuando cambia)
`const`	No	Bloque	Sí (por defecto)

Tipos de datos

JavaScript tiene varios tipos de datos. Diego los fue escribiendo uno por uno en la consola mientras Camila observaba.

String (cadena de texto)

```
let nombre = "Camila";
let apellido = 'Ríos';
let mensaje = `Bienvenida, ${nombre}!`; // Template string
```

Las **template strings** (con backticks) permiten incrustar variables dentro del texto usando `\${}`.

Number (número)

```
let edad = 27;
let precio = 29.99;
let temperatura = -5;
```

JavaScript no diferencia entre enteros y decimales: todo es `number`.

Boolean (booleano)

```
let estaLogueado = true;
let esMayorDeEdad = false;
```

Solo dos valores: `true` (verdadero) o `false` (falso).

Null (nulo)

```
let seleccion = null; // Intencionalmente vacío
```

Representa la ausencia intencional de un valor.

Undefined (indefinido)

```
let curso;
console.log(curso); // undefined
```

Significa que una variable ha sido declarada pero no tiene valor asignado.

typeof

El operador `typeof` te dice el tipo de un valor:

```
console.log(typeof "Hola"); // "string"
console.log(typeof 42); // "number"
console.log(typeof true); // "boolean"
console.log(typeof null); // "object" (error histórico de JS)
console.log(typeof undefined); // "undefined"
```

Comentarios

—El código se escribe para las máquinas —dijo Diego—, pero se lee para los humanos. Los **comentarios** te ayudan a explicar qué hace tu código.

```
// Esto es un comentario de una línea

/*
  Esto es un comentario
```

```
de varias líneas
muy útil para explicar
bloques de código
*/
```

Los comentarios no se ejecutan. Son solo para quien lea el código (incluyendo tu yo del futuro).

El primer problema de Camila

Camila necesitaba mostrar el nombre del usuario en la plataforma. Diego la guió:

```
const usuario = "Camila";
let saludo = "Bienvenida a AprendeYa, " + usuario + "!";
console.log(saludo);

// Con template strings es más fácil:
let saludoModerno = `Bienvenida a AprendeYa, ${usuario}!`;
console.log(saludoModerno);
```

—¿Ves la diferencia? —preguntó Diego—. Las **template strings** hacen el código más legible.

—¿Y si el usuario no está logueado? —preguntó Camila.

—Buena pregunta. Eso lo veremos cuando aprendamos condicionales. Por ahora, guardemos el nombre del usuario en una variable y mostrémoslo.

Camila abrió su editor y creó su primer archivo: `app.js`. Escribió:

```
// Datos del usuario
const nombreUsuario = "Camila Ríos";
let edadUsuario = 27;
const esEstudiante = true;

// Mensaje de bienvenida
console.log(`Hola, ${nombreUsuario}!`);
console.log(`Edad: ${edadUsuario}`);
console.log(`¿Es estudiante? ${esEstudiante}`);
```

—Perfecto —dijo Diego—. Acabas de crear tu primer script con variables y tipos de datos.

Enigmas del Capítulo 1

Enigma 1.1: Tu primer programa

Abre la consola del navegador y escribe un programa que:

1. Declare una variable `const` con tu nombre
2. Declare una variable `let` con tu edad
3. Declare una variable `const` con tu lenguaje favorito (por ahora, JavaScript)
4. Muestre un saludo en consola usando template strings

Enigma 1.2: Detecta el tipo

Usa `typeof` para determinar el tipo de los siguientes valores. Escribe en consola:

```
typeof "42"  
typeof 42  
typeof true  
typeof null  
typeof undefined  
typeof "true"
```

¿Hay algún resultado que te sorprenda?

Enigma 1.3: El mensaje de la plataforma

Crea una variable `const` llamada `curso` con el valor "JavaScript Básico". Crea otra variable `let` llamada `progreso` con el valor 15. Usa template strings para mostrar en consola: "Estás tomando el curso JavaScript Básico. Tu progreso es 15%."

Enigma 1.4: Conversión de tipos

¿Qué crees que imprimen estos `console.log`? Escríbelos en la consola para verificarlo:

```
console.log("5" + 3);  
console.log("5" - 3);  
console.log("5" * "3");  
console.log("Hola" + 5);
```

Lo que aprendiste

- **JavaScript** es el lenguaje que da interactividad a la web
- La **consola del navegador** es tu herramienta principal de depuración
- Las **variables** guardan datos: `const` (constante) y `let` (variable)
- Los **tipos de datos** principales: string, number, boolean, null, undefined
- `typeof` te dice el tipo de un valor
 - **Template strings** con backticks y ``${}`` para interpolar variables
 - Los **comentarios** hacen tu código más legible

Camila guardó su archivo y sonrió. Había escrito su primer código JavaScript. No era mucho, pero era el primer paso de un viaje que cambiaría la forma en que veía la web.

—Mañana —dijo Diego, levantándose—, vamos a hacer que la plataforma tome decisiones. Prepárate.

Capítulo 2: Tomando Decisiones

Operadores y Condicionales

—Buenos días —dijo Diego, dejando una taza de café en el escritorio de Camila—. Hoy vamos a hacer que la plataforma piense.

—¿Piense? —Camila levantó una ceja.

—Sí. Hasta ahora solo guardamos datos. Pero un programa sin decisiones es como un semáforo que siempre muestra verde: inútil. Necesitamos **condicionales** para que el código reaccione según el contexto.

Camila abrió su editor. Tenía lista la estructura de la plataforma: un menú, una sección de cursos, un área de perfil. Pero todo se mostraba igual para todos los usuarios.

—El primer problema —dijo Diego—: necesitamos saber si el usuario está logueado para mostrar contenido personalizado.

Operadores aritméticos

—Antes de las decisiones, necesitamos las herramientas para comparar y calcular —Diego escribió en la consola:

```
// Operadores aritméticos básicos
let suma = 10 + 5;      // 15
let resta = 10 - 5;     // 5
let multiplicacion = 10 * 5; // 50
let division = 10 / 5;  // 2
let modulo = 10 % 3;    // 1 (resto de la división)
```

—El operador `%` (módulo) es útil para saber si un número es par o impar, o para ciclos —explicó Diego.

Operadores de comparación

—Para tomar decisiones, necesitamos comparar valores.

```
// Comparación
console.log(5 == "5"); // true (compara solo valor)
console.log(5 === "5"); // false (compara valor y tipo)
```

```
console.log(5 != "5"); // false
console.log(5 !== "5"); // true
console.log(10 > 5); // true
console.log(10 < 5); // false
console.log(10 >= 10); // true
console.log(10 <= 5); // false
```

—La regla de oro —dijo Diego—: **siempre usa `===` y `!==`**. El `==` y `!=` hacen conversiones automáticas que pueden causar bugs sutiles.

Operador	Significado	Ejemplo true
`===`	Estrictamente igual	`5 === 5`
`!==`	Estrictamente diferente	`5 !== "5"``
`>`	Mayor que	`10 > 5`
`<`	Menor que	`3 < 7`
`>=`	Mayor o igual	`5 >= 5`
`<=`	Menor o igual	`3 <= 5`

Operadores lógicos

—Ahora —continuó Diego—, ¿qué pasa si necesitamos verificar varias condiciones a la vez?

```
let edad = 18;
let tienePermiso = true;

// AND (&&): ambas deben ser true
console.log(edad >= 18 && tienePermiso); // true

// OR (||): al menos una debe ser true
console.log(edad >= 18 || tienePermiso); // true

// NOT (!): invierte el valor
console.log(!true); // false
console.log(!false); // true
```

Diego dibujó una tabla en su cuaderno:

```
// Tabla de verdad de AND (&&)
console.log(true && true); // true
console.log(true && false); // false
console.log(false && true); // false
console.log(false && false); // false

// Tabla de verdad de OR (||)
console.log(true || true); // true
console.log(true || false); // true
console.log(false || true); // true
console.log(false || false); // false
```

If / Else if / Else

—Ahora sí —dijo Diego—. Vamos a tomar decisiones.

La primera función que Camila necesitaba: verificar si un usuario está logueado y mostrar contenido diferente.

```
const usuarioLogueado = true;
const nombreUsuario = "Camila";

if (usuarioLogueado) {
  console.log(`Bienvenida de nuevo, ${nombreUsuario}!`);
} else {
  console.log("Bienvenido. Por favor, inicia sesión.");
}
```

—¿Y si necesito más opciones? —preguntó Camila.

—Usas `else if`:

```
const rol = "admin";

if (rol === "admin") {
  console.log("Acceso total a la plataforma");
} else if (rol === "instructor") {
  console.log("Acceso para crear y editar cursos");
} else if (rol === "estudiante") {
  console.log("Acceso para ver cursos y progreso");
} else {
  console.log("Rol no reconocido");
}
```

Camila aplicó esto a su plataforma:

```
const usuario = {
  nombre: "Camila Ríos",
  logueado: true,
  rol: "admin"
};

if (usuario.logueado) {
  console.log(`Hola, ${usuario.nombre}`);
  if (usuario.rol === "admin") {
    console.log("Tienes permisos de administrador");
  }
} else {
  console.log("Por favor, inicia sesión");
}
```

—¡Un `if` dentro de otro `if`! —exclamó Camila.

—Se llama **anidamiento**. Úsalo con moderación; demasiados niveles hacen el código difícil de leer.

Switch

—Cuando tienes muchas condiciones basadas en un mismo valor, `switch` puede ser más limpio:

```
const dia = "lunes";

switch (dia) {
  case "lunes":
    console.log("Inicio de semana. Nuevo curso disponible.");
    break;
  case "martes":
  case "miercoles":
  case "jueves":
    console.log("Día de estudio. Sigue tu progreso.");
    break;
  case "viernes":
    console.log("Viernes de repaso. Prepara tus dudas.");
    break;
  case "sabado":
  case "domingo":
    console.log("Fin de semana. Disfruta y aprende algo nuevo.");
    break;
  default:
    console.log("Día no válido");
}
```

—El `break` es importante —explicó Diego—. Sin él, el código sigue ejecutando los casos siguientes aunque ya haya encontrado una coincidencia. Eso se llama **fall-through**.

Operador ternario

—Hay un atajo para condicionales simples —dijo Diego—. El **operador ternario**:

```
// Sintaxis: condición ? valorSiTrue : valorSiFalse

const edad = 20;
const mensaje = edad >= 18 ? "Eres mayor de edad" : "Eres menor de edad";
console.log(mensaje);

// Es equivalente a:
let mensajeLargo;
if (edad >= 18) {
  mensajeLargo = "Eres mayor de edad";
} else {
  mensajeLargo = "Eres menor de edad";
}
```

—Úsalo para condicionales simples de una línea. Para lógica más compleja, usa `if/else`.

El problema real de Camila

La plataforma necesitaba mostrar el progreso del usuario con un mensaje personalizado según el porcentaje:

```
const progreso = 65;
let mensajeProgreso;

if (progreso === 0) {
  mensajeProgreso = "Aún no has empezado el curso. ¡Ánimate!";
} else if (progreso < 25) {
  mensajeProgreso = "Has comenzado. ¡Sigue así!";
} else if (progreso < 50) {
  mensajeProgreso = "Ya llevas menos de la mitad. ¡No te rindas!";
} else if (progreso < 75) {
  mensajeProgreso = "Más de la mitad. ¡Estás avanzando!";
} else if (progreso < 100) {
  mensajeProgreso = "Casi llegas. ¡Finaliza el curso!";
} else {
  mensajeProgreso = "¡Felicidades! Has completado el curso.";
}

console.log(mensajeProgreso);
```

—¿Y si el progreso es negativo o mayor a 100? —preguntó Camila.

—Buena observación. Validar los datos antes de usarlos es una buena práctica:

```
function obtenerMensajeProgreso(progreso) {
  if (progreso < 0 || progreso > 100) {
    return "Progreso inválido";
  }

  if (progreso === 0) return "Aún no has empezado";
  if (progreso < 25) return "Has comenzado. ¡Sigue así!";
  if (progreso < 50) return "Ya llevas menos de la mitad";
  if (progreso < 75) return "Más de la mitad. ¡Estás avanzando!";
  if (progreso < 100) return "Casi llegas";
  return "¡Felicidades! Has completado el curso.";
}

console.log(obtenerMensajeProgreso(65));
```

—Las funciones las veremos pronto —dijo Diego—. Pero mira cómo el código se vuelve más legible cuando validas primero.

Enigmas del Capítulo 2

Enigma 2.1: Calculadora de notas

Crea una variable `let nota = 85`. Usando condicionales, muestra en consola:

- "Excelente" si la nota es ≥ 90

- "Bien" si la nota es ≥ 70 y < 90
- "Regular" si la nota es ≥ 50 y < 70
- "Necesitas mejorar" si la nota es < 50

Enigma 2.2: Acceso a la plataforma

Tres variables: ``logueado`` (boolean), ``tieneSuscripcion`` (boolean), ``esAdmin`` (boolean).

Escribe un condicional que:

- Si está logueado Y (tiene suscripción O es admin): "Acceso concedido"
- Si está logueado pero no tiene suscripción ni es admin: "Debes adquirir una suscripción"
- Si no está logueado: "Por favor, inicia sesión"

Enigma 2.3: Día laboral o finde

Usa ``switch`` con una variable ``const dia = "sabado"``. Muestra si es "Día laboral" (lunes a viernes) o "Fin de semana" (sábado, domingo).

Enigma 2.4: ¿Par o impar?

Usa el operador módulo ``%`` para determinar si un número es par o impar. Prueba con: ``15``, ``22``, ``7``, ``100``. Muestra en consola: "El número X es par" o "El número X es impar".

Lo que aprendiste

- **Operadores aritméticos:** ``+``, ``-``, ``*``, ``/``, ``%``
 - **Operadores de comparación:** ``==``, ``!=``, ``>``, ``<``, ``>=``, ``<=``
 - ``if/else if/else``: Para decisiones con múltiples caminos
 - ``switch``: Alternativa limpia para múltiples casos
 - **Operador ternario:** ``condición ? valorTrue : valorFalse``
 - La importancia de **validar datos** antes de procesarlos
-

—Buen trabajo —dijo Diego—. Ahora la plataforma ya puede distinguir entre un usuario logueado y uno que no lo está. Mañana veremos cómo organizar todo este código para no repetirnos.

Camila guardó su archivo. Por primera vez, sintió que el código no era solo instrucciones, sino una conversación: si esto, entonces aquello. Como la vida misma.

Capítulo 3: Las Funciones del Conocimiento

Funciones

Camila tenía un problema. Estaba copiando y pegando el mismo código en varias partes de su archivo.

—Mira esto —le dijo a Diego, señalando la pantalla:

```
// Validar usuario
if (usuario.logueado) {
  console.log("Bienvenido, " + usuario.nombre);
}

// En otra parte del código...
if (otroUsuario.logueado) {
  console.log("Bienvenido, " + otroUsuario.nombre);
}

// Y otra vez...
if (admin.logueado) {
  console.log("Bienvenido, " + admin.nombre);
}
```

—Estás repitiendo la misma lógica —dijo Diego—. En programación, esto se llama **código duplicado** y es malo. ¿Por qué?

1. Si necesitas cambiar el mensaje de bienvenida, tienes que cambiarlo en tres lugares.
2. Es más fácil cometer errores.
3. El código es más largo de lo necesario.

La solución: **funciones**.

Funciones declaradas

—Una función es un bloque de código reutilizable que hace una tarea específica.

```
// Declarar una función
function saludar(nombre) {
```

```

    console.log(`Hola, ${nombre}! Bienvenido a AprendeYa.`);
  }

  // Llamar (ejecutar) la función
  saludar("Camila");
  saludar("Diego");
  saludar("Valeria");

```

—La función `saludar` recibe un **parámetro** (`nombre`) y ejecuta el código dentro de las llaves. Cada vez que la llamas, se ejecuta con el valor que le pases.

Parámetros y argumentos

```

function sumar(a, b) {
  return a + b;
}

let resultado = sumar(5, 3);
console.log(resultado); // 8

```

- `a` y `b` son **parámetros** (los nombres que usa la función)
- `5` y `3` son **argumentos** (los valores reales que pasas)

Return

La palabra clave `return` devuelve un valor desde la función. Sin `return`, la función devuelve `undefined`.

```

function multiplicar(x, y) {
  return x * y;
  console.log("Esto nunca se ejecuta"); // Código muerto
}

let producto = multiplicar(4, 5);
console.log(producto); // 20

```

—Todo lo que esté después de `return` no se ejecuta —explicó Diego—. El `return` termina la función.

Funciones expresadas

—Hay otra forma de crear funciones: asignarlas a una variable.

```

// Función expresada
const saludar = function(nombre) {
  return `Hola, ${nombre}!`;
};

console.log(saludar("Camila")); // "Hola, Camila!"

```

La diferencia principal: las **funciones declaradas** pueden llamarse antes de su declaración (hoisting), las **funciones expresadas** no.

```
// Esto funciona (función declarada)
console.log(despedir("Camila"));
function despedir(nombre) {
  return `Adiós, ${nombre}!`;
}

// Esto da error (función expresada)
console.log(despedir2("Camila")); // ReferenceError
const despedir2 = function(nombre) {
  return `Adiós, ${nombre}!`;
};
```

Arrow functions

—JavaScript moderno introdujo las **arrow functions** —dijo Diego—. Son una forma más corta de escribir funciones.

```
// Función tradicional
const sumarTradicional = function(a, b) {
  return a + b;
};

// Arrow function
const sumarFlecha = (a, b) => {
  return a + b;
};

// Arrow function aún más corta (return implícito)
const sumarCorta = (a, b) => a + b;

console.log(sumarCorta(5, 3)); // 8
```

Si la arrow function tiene un solo parámetro, se pueden omitir los paréntesis:

```
const cuadrado = x => x * x;
console.log(cuadrado(4)); // 16
```

Si no tiene parámetros, se usan paréntesis vacíos:

```
const saludar = () => "Hola, mundo!";
```

—Las arrow functions son muy usadas en JavaScript moderno —dijo Diego—. Las verás mucho en arrays, eventos y promesas.

Scope (ámbito)

—¿Dónde vive una variable? —preguntó Diego—. Eso es el **scope**.

Scope global

Las variables declaradas fuera de cualquier función son globales: accesibles desde cualquier

parte.

```
const nombreGlobal = "Soy global";

function mostrarNombre() {
  console.log(nombreGlobal); // Accesible
}

mostrarNombre();
console.log(nombreGlobal); // Accesible
```

Scope local (de función)

Las variables declaradas dentro de una función solo existen dentro de ella.

```
function crearMensaje() {
  const mensajeLocal = "Soy local";
  console.log(mensajeLocal); // Accesible aquí
}

crearMensaje();
console.log(mensajeLocal); // ReferenceError: no existe fuera
```

Scope de bloque (let y const)

`let` y `const` tienen scope de bloque (entre llaves `{}`), a diferencia de `var`.

```
if (true) {
  var vieja = "Soy var"; // Scope global
  let moderna = "Soy let"; // Scope de bloque
  const constante = "Soy const"; // Scope de bloque
}

console.log(vieja); // "Soy var" (accesible)
console.log(moderna); // ReferenceError
console.log(constante); // ReferenceError
```

Hoisting

—JavaScript tiene un comportamiento peculiar llamado **hoisting** (elevación). Las declaraciones de funciones y `var` se "elevan" al inicio de su contexto.

```
// Esto funciona por hoisting
console.log(sumar(2, 3)); // 5

function sumar(a, b) {
  return a + b;
}
```

Con `var`, la declaración se eleva pero la asignación no:

```
console.log(nombre); // undefined (no error)
var nombre = "Camila";
```

```
// Es como si JS interpretara:  
var nombre;  
console.log(nombre); // undefined  
nombre = "Camila";
```

Con `let` y `const`, el hoisting existe pero no se puede acceder a la variable antes de su declaración (Temporal Dead Zone):

```
console.log(edad); // ReferenceError  
let edad = 27;
```

Organizando la plataforma con funciones

Camila aplicó lo aprendido para organizar su código:

```
// Funciones de utilidad  
function obtenerSaludo(nombre, hora) {  
  if (hora < 12) return `Buenos días, ${nombre}!`;   
  if (hora < 18) return `Buenas tardes, ${nombre}!`;   
  return `Buenas noches, ${nombre}!`;   
}  
  
function puedeAccederAlCurso(usuario, curso) {  
  if (!usuario.logueado) return false;  
  if (curso.gratuito) return true;  
  return usuario.suscripcionActiva;  
}  
  
function calcularProgreso(leccionesCompletadas, totalLecciones) {  
  if (totalLecciones === 0) return 0;  
  return Math.round((leccionesCompletadas / totalLecciones) * 100);  
}  
  
// Uso de las funciones  
const camila = { nombre: "Camila Ríos", logueado: true, suscripcionActiva: true };  
const cursoJS = { titulo: "JavaScript Básico", gratuito: false };  
  
console.log(obtenerSaludo(camila.nombre, 10)); // "Buenos días, Camila Ríos!"  
console.log(puedeAccederAlCurso(camila, cursoJS)); // true  
console.log(calcularProgreso(3, 10)); // 30
```

—Ahora el código es más legible, reutilizable y fácil de mantener —dijo Diego—. Si necesitas cambiar el saludo, solo lo haces en un lugar.

Enigmas del Capítulo 3

Enigma 3.1: Tu primera función

Crea una función llamada `esMayorDeEdad` que reciba una edad y devuelva `true` si es mayor

o igual a 18, ``false`` en caso contrario. Prueba con 15, 18, 21 y 0.

Enigma 3.2: Calculadora básica

Crea tres funciones: ``sumar(a, b)``, ``restar(a, b)``, ``multiplicar(a, b)``. Luego crea una función ``calculadora(operacion, a, b)`` que use las funciones anteriores según el valor de ``operacion`` ("suma", "resta", "multiplica").

Enigma 3.3: Validación de formulario

Crea una función ``validarEmail(email)`` que:

- Devuelva ``true`` si el email contiene un ``@`` y un ``.`` después del ``@``
- Devuelva ``false`` en caso contrario

Usa ``includes()`` para verificar. Prueba con: "camila@email.com", "invalido@", "sinarroba.com".

Enigma 3.4: Conversión de temperatura

Escribe una función flecha ``celsiusAFahrenheit(celsius)`` que convierta grados Celsius a Fahrenheit usando la fórmula: ``F = C * 9/5 + 32``. Prueba con 0 (debe dar 32) y 100 (debe dar 212).

Lo que aprendiste

- Las **funciones** encapsulan código reutilizable
 - **Parámetros** son los nombres; **argumentos** son los valores
 - ``return`` devuelve un valor y termina la función
 - **Funciones declaradas** vs **funciones expresadas**
 - **Arrow functions**: sintaxis moderna y concisa ``() => {}``
 - **Scope global** vs **scope local** vs **scope de bloque**
 - **Hoisting**: elevación de declaraciones
 - Las funciones hacen el código más **modular** y **mantenible**
-

Diego se levantó y estiró los brazos.

—Mañana vamos a manejar listas de cursos y estudiantes. Arrays y objetos. Ahí las funciones se vuelven aún más poderosas.

Camila miró su código. Ahora tenía funciones limpias y reutilizables. La plataforma comenzaba a tener estructura. Y ella comenzaba a pensar como programadora.

Capítulo 4: Listas y Colecciones

Arrays y Objetos

La plataforma "AprendeYa" tenía un catálogo de cursos. Camila los había diseñado en Figma, pero en el código eran solo texto estático en HTML.

—Necesitamos manejar listas de cursos, estudiantes y progreso —dijo Diego—. Para eso existen los **arrays** y los **objetos**.

Arrays

—Un **array** es una lista ordenada de elementos. Piensa en una caja con compartimentos numerados.

```
// Crear un array
const cursos = ["HTML Básico", "CSS Avanzado", "JavaScript Básico"];
const numeros = [1, 2, 3, 4, 5];
const mixto = ["Texto", 42, true, null, [1, 2, 3]];
const vacio = [];
```

Acceder a elementos

Los arrays usan **índices** que empiezan desde 0.

```
const frutas = ["manzana", "banana", "naranja"];

console.log(frutas[0]); // "manzana"
console.log(frutas[1]); // "banana"
console.log(frutas[2]); // "naranja"
console.log(frutas[3]); // undefined (no existe)
```

Propiedad length

```
console.log(frutas.length); // 3

const ultimoElemento = frutas[frutas.length - 1];
console.log(ultimoElemento); // "naranja"
```

Métodos básicos

```
const cursos = ["HTML", "CSS"];

// push: agrega al final
cursos.push("JavaScript");
console.log(cursos); // ["HTML", "CSS", "JavaScript"]

// pop: elimina el último
const ultimo = cursos.pop();
console.log(ultimo); // "JavaScript"
console.log(cursos); // ["HTML", "CSS"]

// unshift: agrega al inicio
cursos.unshift("Python");
console.log(cursos); // ["Python", "HTML", "CSS"]

// shift: elimina el primero
const primero = cursos.shift();
console.log(primero); // "Python"
console.log(cursos); // ["HTML", "CSS"]
```

Métodos de iteración

—Estos son los más poderosos —dijo Diego—. Te permiten recorrer arrays sin usar bucles tradicionales.

forEach: ejecuta una función para cada elemento

```
const cursos = ["HTML", "CSS", "JavaScript"];

cursos.forEach(function(curso, indice) {
  console.log(`${indice + 1}. ${curso}`);
});
// "1. HTML"
// "2. CSS"
// "3. JavaScript"
```

map: crea un nuevo array transformando cada elemento

```
const numeros = [1, 2, 3, 4, 5];
const cuadrados = numeros.map(function(num) {
  return num * num;
});
console.log(cuadrados); // [1, 4, 9, 16, 25]

// Con arrow function:
const dobles = numeros.map(n => n * 2);
console.log(dobles); // [2, 4, 6, 8, 10]
```

filter: crea un nuevo array con los elementos que cumplen una condición

```
const edades = [15, 22, 18, 30, 12, 17, 25];
const mayoresDeEdad = edades.filter(edad => edad >= 18);
console.log(mayoresDeEdad); // [22, 18, 30, 25]
```

find: encuentra el primer elemento que cumple una condición

```
const usuarios = ["Ana", "Carlos", "María", "Pedro"];
const encontrado = usuarios.find(nombre => nombre.startsWith("M"));
console.log(encontrado); // "María"
```

Objetos

—Si los arrays son listas ordenadas, los **objetos** son colecciones de propiedades con nombre.

```
// Crear un objeto
const curso = {
  titulo: "JavaScript Básico",
  duracion: "10 horas",
  nivel: "Principiante",
  gratuito: false
};

// Acceder a propiedades (notación de punto)
console.log(curso.titulo); // "JavaScript Básico"
console.log(curso.nivel);  // "Principiante"

// Acceder con corchetes
console.log(curso["titulo"]); // "JavaScript Básico"

// Modificar propiedades
curso.gratuito = true;
console.log(curso.gratuito); // true

// Agregar nuevas propiedades
curso.rating = 4.5;
console.log(curso.rating); // 4.5
```

Objetos anidados

—Los objetos pueden contener otros objetos y arrays.

```
const estudiante = {
  nombre: "Camila Ríos",
  edad: 27,
  cursos: ["HTML Básico", "CSS Avanzado"],
  direccion: {
    ciudad: "Lima",
    pais: "Perú"
  },
  progreso: {
    "HTML Básico": 100,
    "CSS Avanzado": 70
  }
};
```

```
console.log(estudiante.nombre);           // "Camila Ríos"
console.log(estudiante.cursos[0]);        // "HTML Básico"
console.log(estudiante.direccion.ciudad); // "Lima"
console.log(estudiante.progreso["HTML Básico"]); // 100
```

Métodos en objetos

—Los objetos también pueden tener funciones como propiedades. Se llaman **métodos**.

```
const usuario = {
  nombre: "Camila",
  edad: 27,
  saludar: function() {
    return `Hola, soy ${this.nombre}`;
  },
  // Método con arrow function
  mostrarEdad: () => {
    return `Tengo ${usuario.edad} años`;
  }
};

console.log(usuario.saludar()); // "Hola, soy Camila"
console.log(usuario.mostrarEdad()); // "Tengo 27 años"
```

—La palabra clave `this` dentro de un método se refiere al objeto que contiene el método.

JSON

—Hay un formato muy usado para intercambiar datos: **JSON** (JavaScript Object Notation). Es básicamente texto con la misma estructura que los objetos de JavaScript.

```
// Objeto JavaScript
const cursoJS = {
  titulo: "JavaScript Básico",
  precio: 0,
  lecciones: 15
};

// Convertir a JSON (string)
const cursoJSON = JSON.stringify(cursoJS);
console.log(cursoJSON);
// '{"titulo":"JavaScript Básico","precio":0,"lecciones":15}'

// Convertir de JSON a objeto
const cursoObjeto = JSON.parse(cursoJSON);
console.log(cursoObjeto.titulo); // "JavaScript Básico"
```

—JSON es el lenguaje universal de intercambio de datos en la web. Lo usaremos en el capítulo 9 con APIs.

Catálogo de cursos de AprendeYa

Camila organizó todo el catálogo de la plataforma:

```
const catalogo = [
  {
    id: 1,
    titulo: "HTML para Principiantes",
    duracion: "5 horas",
    nivel: "principiante",
    gratuito: true,
    rating: 4.8
  },
  {
    id: 2,
    titulo: "CSS Avanzado",
    duracion: "8 horas",
    nivel: "intermedio",
    gratuito: false,
    rating: 4.5
  },
  {
    id: 3,
    titulo: "JavaScript Básico",
    duracion: "10 horas",
    nivel: "principiante",
    gratuito: true,
    rating: 4.9
  },
  {
    id: 4,
    titulo: "React: Tu primera app",
    duracion: "12 horas",
    nivel: "avanzado",
    gratuito: false,
    rating: 4.2
  }
];

// Filtrar cursos gratuitos
const cursosGratis = catalogo.filter(curso => curso.gratuito);
console.log(cursosGratis);
// Devuelve los cursos con gratuito: true

// Obtener solo los títulos
const titulos = catalogo.map(curso => curso.titulo);
console.log(titulos);
// ["HTML para Principiantes", "CSS Avanzado", "JavaScript Básico", "React: Tu primera app"]

// Encontrar un curso por ID
const cursoJS = catalogo.find(curso => curso.id === 3);
console.log(cursoJS.titulo); // "JavaScript Básico"
```

```
// Calcular rating promedio
const promedio = catalogo
  .map(curso => curso.rating)
  .reduce((total, rating) => total + rating, 0) / catalogo.length;
console.log(promedio.toFixed(2)); // "4.60"
```

—`reduce` es otro método poderoso —dijo Diego—. Reduce un array a un solo valor. Aprenderás a usarlo con la práctica.

Enigmas del Capítulo 4

Enigma 4.1: Tu lista de compras

Crea un array llamado `listaCompras` con 5 productos. Agrega un producto al final con `push`, elimina el primero con `shift`, y muestra el array final y su longitud.

Enigma 4.2: Catálogo de cursos

Crea un array de objetos con 3 cursos (cada uno con `titulo`, `duracion`, `precio`). Usa `map` para obtener un array solo con los títulos. Usa `filter` para encontrar los cursos gratis (precio === 0).

Enigma 4.3: Objeto estudiante

Crea un objeto `estudiante` con: `nombre`, `edad`, `cursos` (array), `direccion` (objeto con `ciudad`, `pais`). Agrega un método `presentarse()` que devuelva "Hola, soy [nombre] de [ciudad], [pais]".

Enigma 4.4: Transformando datos

Dado el array `const nums = [5, 12, 8, 21, 3, 15]`:

1. Usa `filter` para obtener los números mayores a 10
 2. Usa `map` para obtener el doble de cada número
 3. Usa `find` para encontrar el primer número mayor a 10
-

Lo que aprendiste

- **Arrays:** listas ordenadas `[elemento1, elemento2]`
 - Índices empiezan en **0**
 - **Métodos:** `push`, `pop`, `shift`, `unshift`, `length`
 - **Iteración:** `forEach`, `map`, `filter`, `find`
 - **Objetos:** colecciones clave-valor `{ clave: valor }`
 - **Propiedades y métodos** en objetos
 - `this` se refiere al objeto contenedor
 - **JSON:** formato de intercambio de datos
-

Camila miró el catálogo de cursos funcionando como un array de objetos. La plataforma

comenzaba a tener datos reales, no solo maquetas.

—Mañana —dijo Diego— vamos a conectar estos datos con la página. Vas a ver cómo el DOM cobra vida.

Camila sonrió. Por primera vez, sentía que estaba construyendo algo real.

Capítulo 5: La Vida en la Página

DOM y Eventos

—Hasta ahora hemos trabajado en la consola —dijo Diego, cerrando la pestaña de herramientas de desarrollador—. Pero la magia real ocurre cuando JavaScript interactúa con la página web.

Camila abrió su archivo HTML de la plataforma. Había una estructura limpia con `<header>`, `<main>`, `<section>` y `<footer>`. Todo se veía hermoso pero inmóvil.

—Vamos a darle vida —dijo Diego.

¿Qué es el DOM?

—**DOM** significa **Document Object Model**. Es una representación de tu página HTML como un árbol de objetos que JavaScript puede manipular.

Cuando el navegador carga tu HTML, crea el DOM. Cada etiqueta HTML se convierte en un **node** que puedes modificar, eliminar o crear.

```
<!DOCTYPE html>
<html>
<head>
  <title>AprendeYa</title>
</head>
<body>
  <h1 id="titulo">Bienvenido a AprendeYa</h1>
  <p class="descripcion">Tu plataforma de cursos online</p>
</body>
</html>
```

—Cada elemento del HTML se convierte en un objeto JavaScript con propiedades y métodos.

Seleccionando elementos del DOM

Para manipular un elemento, primero debes seleccionarlo.

document.getElementById()

```
const titulo = document.getElementById("titulo");
console.log(titulo); // <h1 id="titulo">Bienvenido a AprendeYa</h1>
```

```
console.log(titulo.textContent); // "Bienvenido a AprendeYa"
```

document.querySelector()

—Este es el más versátil —dijo Diego—. Usa selectores CSS.

```
// Selecciona por ID (usa #)
const titulo = document.querySelector("#titulo");

// Selecciona por clase (usa .)
const descripcion = document.querySelector(".descripcion");

// Selecciona por etiqueta
const primerParrafo = document.querySelector("p");

// Selecciona el primer elemento que coincida
const primerBoton = document.querySelector("button");
```

document.querySelectorAll()

```
// Selecciona TODOS los elementos que coincidan
const todosLosParrafos = document.querySelectorAll("p");
const botones = document.querySelectorAll(".btn");

console.log(todosLosParrafos.length); // Número de párrafos
todosLosParrafos.forEach(p => console.log(p.textContent));
```

Método	Devuelve	Usa
<code>getElementById("id")`</code>	Un elemento	IDs únicos
<code>querySelector("selector")`</code>	Primer elemento	Cualquier selector CSS
<code>querySelectorAll("selector")`</code>	NodeList (como array)	Múltiples elementos

Eventos: la página reacciona

—Un **evento** es algo que ocurre en la página: un clic, una tecla presionada, el mouse moviéndose. JavaScript puede "escuchar" esos eventos y reaccionar.

addEventListener

```
const boton = document.querySelector("#btn-iniciar-sesion");

boton.addEventListener("click", function() {
  alert("Has hecho clic en Iniciar Sesión");
});
```

—El primer argumento es el tipo de evento (`"click"`). El segundo es la función que se ejecuta cuando ocurre.

Tipos de eventos comunes

```
const boton = document.querySelector("#miBoton");
```

```
const input = document.querySelector("#miInput");
const parrafo = document.querySelector("#miParrafo");

// Click
boton.addEventListener("click", () => {
  console.log("¡Click!");
});

// Mouse over (mouse encima)
boton.addEventListener("mouseover", () => {
  console.log("Mouse está sobre el botón");
});

// Keydown (tecla presionada)
input.addEventListener("keydown", (event) => {
  console.log(`Tecla presionada: ${event.key}`);
});

// Submit (formulario enviado)
const formulario = document.querySelector("#miFormulario");
formulario.addEventListener("submit", (event) => {
  event.preventDefault(); // Evita que la página se recargue
  console.log("Formulario enviado");
});
```

El objeto event

—Cuando ocurre un evento, JavaScript pasa automáticamente un objeto `event` a la función manejadora. Este objeto contiene información sobre el evento.

```
const boton = document.querySelector("#miBoton");

boton.addEventListener("click", function(event) {
  console.log(event.type);    // "click"
  console.log(event.target);  // El elemento que recibió el click
  console.log(event.clientX); // Posición X del mouse
  console.log(event.clientY); // Posición Y del mouse
});
```

this en eventos

—Dentro de un manejador de eventos con función normal, `this` se refiere al elemento que recibió el evento.

```
const botones = document.querySelectorAll(".btn");

botones.forEach(function(boton) {
  boton.addEventListener("click", function() {
    console.log(this.textContent); // El texto del botón clickeado
    this.style.backgroundColor = "blue";
  });
});
```

```
});
```

—Con arrow functions, `this` no se refiere al elemento, sino al contexto exterior. Si necesitas acceder al elemento, usa `event.target`.

```
botones.forEach(boton => {  
  boton.addEventListener("click", (event) => {  
    console.log(event.target.textContent);  
    event.target.style.backgroundColor = "blue";  
  });  
});
```

El primer botón interactivo de Camila

Camila creó su primera funcionalidad interactiva: un botón que muestra el mensaje de bienvenida personalizado.

HTML:

```
<header>  
  <h1 id="titulo">Bienvenido a AprendeYa</h1>  
  <button id="btn-saludo">Saludar</button>  
  <p id="mensaje"></p>  
</header>
```

JavaScript:

```
const btnSaludo = document.querySelector("#btn-saludo");  
const mensaje = document.querySelector("#mensaje");  
const titulo = document.querySelector("#titulo");  
  
btnSaludo.addEventListener("click", function() {  
  const nombre = "Camila";  
  mensaje.textContent = `¡Hola, ${nombre}! Gracias por usar AprendeYa.`;  
  titulo.style.color = "#2ecc71";  
});
```

—¡Funcionó! —exclamó Camila—. La página cambió cuando hice clic.

—Eso es el DOM en acción —dijo Diego—. JavaScript modificó el contenido y el estilo de la página en tiempo real.

Enigmas del Capítulo 5

Enigma 5.1: Botón que cambia de color

Crea un botón en HTML que, al hacerle clic, cambie su propio color de fondo a un color aleatorio. Pista: usa `Math.floor(Math.random() \* 16777215).toString(16)` para generar un color hexadecimal.

Enigma 5.2: Contador de clics

Crea un botón y un párrafo que muestre un número. Cada clic al botón incrementa el número. Llega al 10 y reinicia.

Enigma 5.3: Mouseover en tarjetas

Crea tres divs con clase "tarjeta". Cuando el mouse pase sobre una, cambia su borde a verde. Cuando salga, vuelve al borde original.

Enigma 5.4: Input en vivo

Crea un input de texto y un párrafo. Cada vez que el usuario escriba en el input, el párrafo muestra en tiempo real lo que se está escribiendo. Usa el evento ``input``.

Lo que aprendiste

- El **DOM** es la representación HTML como objetos JavaScript
 - ``querySelector`` y ``querySelectorAll`` seleccionan elementos
 - ``addEventListener`` escucha eventos
 - Tipos de eventos: ``click``, ``mouseover``, ``keydown``, ``submit``, ``input``
 - El objeto ``event`` contiene información del evento
 - ``this`` en eventos se refiere al elemento (con funciones normales)
 - ``event.target`` es el elemento que desencadenó el evento
-

—Ahora la página ya responde —dijo Diego, guardando los cambios—. Mañana vamos a aprender a transformar la interfaz por completo: crear elementos, eliminarlos, cambiar clases y estilos dinámicamente.

Camila hizo clic en su botón de saludo una vez más. Cada clic cambiaba el color del título. Era un detalle pequeño, pero para ella era enorme: había hecho que la página le respondiera.

Capítulo 6: Transformando la Interfaz

Manipulación del DOM

—Ahora que sabes escuchar eventos —dijo Diego—, es hora de transformar la interfaz dinámicamente. Crear menús que se despliegan, contenido que aparece y desaparece, tarjetas que se agregan en tiempo real.

Camila tenía una lista de tareas: el menú de navegación debía colapsarse en móviles, los cursos debían mostrarse dinámicamente, y los usuarios debían poder marcar lecciones como completadas.

—Vamos por partes —dijo Diego.

innerHTML vs textContent

```
const contenedor = document.querySelector("#contenido");

// textContent: solo texto (más seguro, más rápido)
contenedor.textContent = "Hola, mundo!";

// innerHTML: permite HTML (cuidado con inyección)
contenedor.innerHTML = "<h2>Hola, mundo!</h2><p>Esto es HTML</p>";
```

—Usa `textContent` cuando solo necesites texto. Usa `innerHTML` cuando necesites insertar HTML, pero **nunca** con datos del usuario sin sanitizar.

classList: manejando clases CSS

—Manipular clases CSS directamente es una de las operaciones más comunes.

```
const menu = document.querySelector("#menu-lateral");

// Agregar una clase
menu.classList.add("visible");

// Quitar una clase
menu.classList.remove("visible");

// Alternar (agrega si no existe, quita si existe)
```

```

menu.classList.toggle("visible");

// Verificar si tiene una clase
if (menu.classList.contains("visible")) {
  console.log("El menú está visible");
}

```

Camila aplicó esto al menú de navegación:

```

const btnMenu = document.querySelector("#btn-menu");
const nav = document.querySelector("#nav-principal");

btnMenu.addEventListener("click", () => {
  nav.classList.toggle("activo");
});

```

CSS:

```

#nav-principal {
  display: none;
}

#nav-principal.activo {
  display: block;
}

```

Creando y eliminando elementos

—A veces necesitas crear elementos desde JavaScript y agregarlos al DOM.

createElement y appendChild

```

// Crear un nuevo elemento
const nuevaTarjeta = document.createElement("div");
nuevaTarjeta.classList.add("tarjeta");
nuevaTarjeta.textContent = "Curso: JavaScript Básico";

// Agregarlo al DOM
const contenedor = document.querySelector("#lista-cursos");
contenedor.appendChild(nuevaTarjeta);

```

Crear elementos con estructura

```

function crearTarjetaCurso(titulo, nivel, duracion) {
  const tarjeta = document.createElement("div");
  tarjeta.classList.add("tarjeta-curso");

  const h3 = document.createElement("h3");
  h3.textContent = titulo;

  const pNivel = document.createElement("p");
  pNivel.textContent = `Nivel: ${nivel}`;
}

```

```

const pDuracion = document.createElement("p");
pDuracion.textContent = `Duración: ${duracion}`;

tarjeta.appendChild(h3);
tarjeta.appendChild(pNivel);
tarjeta.appendChild(pDuracion);

return tarjeta;
}

// Uso
const contenedor = document.querySelector("#lista-cursos");
const tarjetaJS = crearTarjetaCurso("JavaScript Básico", "Principiante", "10h");
contenedor.appendChild(tarjetaJS);

```

removeChild y remove

```

// Eliminar un elemento
const tarjeta = document.querySelector("#tarjeta-1");
tarjeta.remove(); // Método directo

// O desde el padre
const contenedor = document.querySelector("#lista-cursos");
const tarjeta2 = document.querySelector("#tarjeta-2");
contenedor.removeChild(tarjeta2);

```

Manipulando estilos con style

```

const elemento = document.querySelector("#destacado");

// Cambiar estilos inline
elemento.style.backgroundColor = "#3498db";
elemento.style.color = "white";
elemento.style.fontSize = "20px";
elemento.style.padding = "16px";
elemento.style.borderRadius = "8px";

```

—Las propiedades CSS se escriben en **camelCase** cuando se usan desde JavaScript:

CSS	JavaScript
<code>`background-color`</code>	<code>`style.backgroundColor`</code>
<code>`font-size`</code>	<code>`style.fontSize`</code>
<code>`margin-top`</code>	<code>`style.marginTop`</code>
<code>`border-radius`</code>	<code>`style.borderRadius`</code>

Trabajando con atributos


```

const enlace = document.querySelector("a");

// Obtener atributo
console.log(enlace.getAttribute("href"));

// Establecer atributo
enlace.setAttribute("href", "https://aprendeya.com/cursos");
enlace.setAttribute("target", "_blank");

// Verificar si tiene atributo
if (enlace.hasAttribute("target")) {
  console.log("El enlace abre en nueva pestaña");
}

// Eliminar atributo
enlace.removeAttribute("target");

```

La funcionalidad de mostrar/ocultar

Camila implementó varias funcionalidades en la plataforma:

```

// Mostrar/Ocultar menú de perfil
const btnPerfil = document.querySelector("#btn-perfil");
const menuPerfil = document.querySelector("#menu-perfil");

btnPerfil.addEventListener("click", () => {
  menuPerfil.classList.toggle("oculto");
});

// Cerrar menú al hacer clic fuera
document.addEventListener("click", (event) => {
  if (!btnPerfil.contains(event.target) && !menuPerfil.contains(event.target)) {
    menuPerfil.classList.add("oculto");
  }
});

```

```

// Generar lista de cursos dinámicamente
const cursos = [
  { titulo: "HTML Básico", nivel: "Principiante", duracion: "5h" },
  { titulo: "CSS Avanzado", nivel: "Intermedio", duracion: "8h" },
  { titulo: "JavaScript", nivel: "Principiante", duracion: "10h" },
  { titulo: "React", nivel: "Avanzado", duracion: "12h" }
];

const contenedorCursos = document.querySelector("#lista-cursos");

cursos.forEach(curso => {
  const tarjeta = document.createElement("div");
  tarjeta.classList.add("tarjeta-curso");

  tarjeta.innerHTML = `

```

```

<h3>${curso.titulo}</h3>
<p><strong>Nivel:</strong> ${curso.nivel}</p>
<p><strong>Duración:</strong> ${curso.duracion}</p>
<button class="btn-inscribirse">Inscribirse</button>
`;

contenedorCursos.appendChild(tarjeta);
});

// Agregar evento a cada botón de inscripción
document.querySelectorAll(".btn-inscribirse").forEach(btn => {
  btn.addEventListener("click", (event) => {
    const tarjeta = event.target.closest(".tarjeta-curso");
    const titulo = tarjeta.querySelector("h3").textContent;
    alert(`Te has inscrito en: ${titulo}`);
  });
});

```

—`closest()` busca el ancestro más cercano que coincida con el selector —explicó Diego—. Muy útil cuando trabajas con eventos en elementos creados dinámicamente.

Enigmas del Capítulo 6

Enigma 6.1: Lista de tareas

Crea un HTML con un input y un botón "Agregar". Cada vez que el usuario escribe y presiona el botón, se agrega un nuevo elemento `- ` a una lista `
`. Si el input está vacío, muestra un mensaje de error.

Enigma 6.2: Tarjetas de colores

Crea un botón que, al hacer clic, agregue una tarjeta de color aleatorio al contenedor. Cada tarjeta debe tener un botón "Eliminar" que la quite del DOM.

Enigma 6.3: Modo oscuro

Crea un botón "Modo Oscuro" que alterne las clases del body. Cuando está activo, el fondo es negro y el texto blanco. Cambia también el texto del botón a "Modo Claro".

Enigma 6.4: Galería dinámica

Dado un array de URLs de imágenes (pueden ser placeholder como `https://via.placeholder.com/150`), genera dinámicamente una galería. Cada imagen debe tener un borde que se resalte al pasar el mouse.

Lo que aprendiste

- `textContent`: texto plano; `innerHTML`: HTML (con precaución)
- `classList`: `add`, `remove`, `toggle`, `contains`
- `createElement` y `appendChild`: crear y agregar elementos

- ``remove`` y ``removeChild``: eliminar elementos
 - ``style``: manipular estilos inline (en camelCase)
 - ``getAttribute`` / ``setAttribute``: trabajar con atributos HTML
 - ``closest()``: buscar ancestros de un elemento
-

—Ahora la plataforma ya no es estática —dijo Camila, viendo cómo las tarjetas de cursos aparecían en pantalla—. Los usuarios pueden ver el contenido dinámicamente.

—Y mañana —dijo Diego— van a poder registrarse. Vamos a trabajar con formularios y validación.

Camila sonrió. La plataforma comenzaba a sentirse como una aplicación de verdad.

Capítulo 7: Formularios Inteligentes

Formularios y Validación

—La plataforma necesita un formulario de registro —dijo Marco, el CEO, en la reunión matutina—. Los inversores quieren ver que podemos capturar usuarios.

Camila abrió su editor. Había diseñado un formulario hermoso en Figma: campos para nombre, email, contraseña, y un botón de registro. Pero en el código, el formulario no hacía nada.

—Hoy vamos a solucionar eso —dijo Diego.

El evento submit

—El evento más importante de un formulario es **submit**. Se dispara cuando el usuario intenta enviarlo.

```
<form id="form-registro">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email">

  <label for="password">Contraseña:</label>
  <input type="password" id="password" name="password">

  <button type="submit">Registrarse</button>
</form>
```

```
const formulario = document.querySelector("#form-registro");

formulario.addEventListener("submit", function(event) {
  event.preventDefault(); // ¡Importante! Evita recargar la página
  console.log("Formulario enviado");
});
```

—`preventDefault()` evita el comportamiento por defecto del formulario, que es recargar la página. Sin esto, perderías todos los datos en JavaScript.

Eventos de formulario

input

Se dispara cada vez que el valor del campo cambia.

```
const inputNombre = document.querySelector("#nombre");

inputNombre.addEventListener("input", function() {
  console.log(`Valor actual: ${this.value}`);
});
```

change

Se dispara cuando el campo pierde el foco y su valor ha cambiado.

```
const inputEmail = document.querySelector("#email");

inputEmail.addEventListener("change", function() {
  console.log(`Email cambiado a: ${this.value}`);
});
```

focus y blur

```
inputNombre.addEventListener("focus", function() {
  this.style.borderColor = "#3498db";
});

inputNombre.addEventListener("blur", function() {
  this.style.borderColor = "#ccc";
  validarNombre(this.value);
});
```

Validación básica

—Validar significa verificar que los datos ingresados cumplen ciertas reglas antes de procesarlos.

```
const formulario = document.querySelector("#form-registro");
const errorNombre = document.querySelector("#error-nombre");
const errorEmail = document.querySelector("#error-email");
const errorPassword = document.querySelector("#error-password");

function validarNombre(nombre) {
  if (nombre.trim() === "") {
    errorNombre.textContent = "El nombre es obligatorio";
    return false;
  }
  if (nombre.trim().length < 3) {
    errorNombre.textContent = "El nombre debe tener al menos 3 caracteres";
    return false;
  }
  errorNombre.textContent = "";
}
```

```

    return true;
}

function validarEmail(email) {
    if (email.trim() === "") {
        errorEmail.textContent = "El email es obligatorio";
        return false;
    }
    if (!email.includes("@") || !email.includes(".")) {
        errorEmail.textContent = "Ingresa un email válido";
        return false;
    }
    errorEmail.textContent = "";
    return true;
}

function validarPassword(password) {
    if (password.length < 6) {
        errorPassword.textContent = "La contraseña debe tener al menos 6 caracteres";
        return false;
    }
    errorPassword.textContent = "";
    return true;
}

formulario.addEventListener("submit", function(event) {
    event.preventDefault();

    const nombre = document.querySelector("#nombre").value;
    const email = document.querySelector("#email").value;
    const password = document.querySelector("#password").value;

    const nombreValido = validarNombre(nombre);
    const emailValido = validarEmail(email);
    const passwordValida = validarPassword(password);

    if (nombreValido && emailValido && passwordValida) {
        console.log("Registro exitoso!", { nombre, email, password });
        // Aquí iría el código para enviar los datos al servidor
    } else {
        console.log("Corrige los errores del formulario");
    }
});

```

Expresiones regulares básicas

—Para validaciones más potentes, usamos **expresiones regulares (regex)**. Son patrones que describen cadenas de texto.

```
// Validar email con regex
```

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function validarEmailRegex(email) {
  return emailRegex.test(email);
}

console.log(validarEmailRegex("camila@email.com")); // true
console.log(validarEmailRegex("invalido@")); // false
```

Patrones útiles:

```
// Solo letras (incluyendo espacios)
const soloLetras = /^[a-zA-ZáéíóúÁÉÍÓÚñÑ\s]+$/;

// Solo números
const soloNumeros = /^\d+$/;

// Teléfono (formato: 999-999-999)
const telefono = /^\d{3}-\d{3}-\d{3}$/;

// URL básica
const url = /^https?:\/\/\./+$/;

// Probar
console.log(soloLetras.test("Camila Ríos")); // true
console.log(soloLetras.test("Camila123")); // false
```

Validación en tiempo real

—Lo más profesional es validar mientras el usuario escribe, no solo al enviar.

```
const inputNombre = document.querySelector("#nombre");
const inputEmail = document.querySelector("#email");
const inputPassword = document.querySelector("#password");

inputNombre.addEventListener("blur", function() {
  validarNombre(this.value);
});

inputEmail.addEventListener("input", function() {
  validarEmail(this.value);
});

inputPassword.addEventListener("input", function() {
  validarPassword(this.value);
});

// También validar al escribir en el nombre
inputNombre.addEventListener("input", function() {
  if (this.value.length >= 3) {
    errorNombre.textContent = "";
  }
});
```

```
}  
});
```

Mostrar errores visualmente

```
function mostrarError(input, mensaje) {  
  const errorDiv = input.nextElementSibling;  
  errorDiv.textContent = mensaje;  
  input.classList.add("input-error");  
  input.classList.remove("input-success");  
}  
  
function mostrarExito(input) {  
  const errorDiv = input.nextElementSibling;  
  errorDiv.textContent = "";  
  input.classList.remove("input-error");  
  input.classList.add("input-success");  
}
```

CSS asociado:

```
.input-error {  
  border: 2px solid #e74c3c;  
  background-color: #fdf0ef;  
}  
  
.input-success {  
  border: 2px solid #2ecc71;  
  background-color: #f0fdf4;  
}  
  
.error-mensaje {  
  color: #e74c3c;  
  font-size: 0.85em;  
  margin-top: 4px;  
}
```

El formulario de registro de AprendeYa

Camila integró todo en un formulario completo:

```
const formulario = document.querySelector("#form-registro");  
  
// Validación en tiempo real  
document.querySelectorAll("#form-registro input").forEach(input => {  
  input.addEventListener("blur", function() {  
    validarCampo(this);  
  });  
  
  input.addEventListener("input", function() {  
    if (this.classList.contains("input-error")) {
```



```

        validarCampo(this);
    }
});
});

function validarCampo(input) {
    const valor = input.value.trim();
    const errorDiv = input.nextElementSibling;

    if (input.required && valor === "") {
        mostrarError(input, "Este campo es obligatorio");
        return false;
    }

    if (input.type === "email") {
        const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailRegex.test(valor)) {
            mostrarError(input, "Ingresa un email válido");
            return false;
        }
    }

    if (input.type === "password" && valor.length < 6) {
        mostrarError(input, "Mínimo 6 caracteres");
        return false;
    }

    mostrarExito(input);
    return true;
}

formulario.addEventListener("submit", function(event) {
    event.preventDefault();

    const inputs = this.querySelectorAll("input");
    let formularioValido = true;

    inputs.forEach(input => {
        if (!validarCampo(input)) {
            formularioValido = false;
        }
    });

    if (formularioValido) {
        const datos = {
            nombre: document.querySelector("#nombre").value,
            email: document.querySelector("#email").value
        };
        console.log("Usuario registrado:", datos);
        alert("¡Registro exitoso! Bienvenido a AprendeYa.");
        this.reset(); // Limpiar formulario
    } else {

```

```
    alert("Corrige los errores antes de continuar");
  }
});
```

Enigmas del Capítulo 7

Enigma 7.1: Validación de edad

Agrega un campo "edad" al formulario que valide que:

- Es obligatorio
- Es un número
- Está entre 18 y 120 años

Enigma 7.2: Confirmación de contraseña

Agrega un campo "confirmar contraseña". Valida que sea igual al campo contraseña antes de enviar el formulario. Muestra error si no coinciden.

Enigma 7.3: Contador de caracteres

Crea un textarea para "biografía" con un contador que muestre los caracteres restantes (máximo 200). El contador debe actualizarse en tiempo real y ponerse rojo cuando se exceda.

Enigma 7.4: Validación de nombre de usuario

Crea una función que valide un nombre de usuario:

- Entre 4 y 15 caracteres
- Solo letras minúsculas, números y guión bajo
- No puede empezar con número

Usa la expresión regular: `/^[a-z][a-z0-9_]{3,14}$/`

Lo que aprendiste

- ``submit``: evento principal de formularios
 - ``preventDefault()``: evita la recarga de página
 - Eventos: ``input`` (tiempo real), ``change`` (al perder foco), ``focus``, ``blur``
 - **Validación**: verificar datos antes de procesarlos
 - **Expresiones regulares (regex)**: patrones para validar texto
 - Validación en **tiempo real** vs validación al enviar
 - Retroalimentación visual con clases CSS
-

Camila probó el formulario una y otra vez. Ingresaba datos inválidos y veía los mensajes de error aparecer. Ingresaba datos correctos y veía los bordes verdes.

—Esto es adictivo —dijo.

—La validación es una de las cosas que más diferencia una página profesional de una amateur

—respondió Diego—. Buen trabajo.

—Mañana —dijo Camila— quiero que los usuarios no pierdan su progreso al cerrar el navegador.

Diego sonrió. —Eso es justo lo que veremos mañana.

Capítulo 8: Guardando el Progreso

Almacenamiento (localStorage)

—Hay un problema —dijo Camila—. Los usuarios pierden su progreso cuando cierran el navegador.

—Exacto —dijo Diego—. Cada vez que recargan la página, todo vuelve a cero. Necesitamos **almacenamiento local**.

¿Qué es localStorage?

localStorage es una API del navegador que permite guardar datos en el equipo del usuario. Los datos persisten incluso después de cerrar el navegador.

Características principales:

- Capacidad: ~5MB por dominio
 - Solo guarda **strings**
 - Los datos no expiran (a menos que se borren manualmente)
 - Especificidad por dominio (cada sitio web tiene su propio localStorage)
-

Métodos de localStorage

setItem: guardar datos

```
// Guardar un valor
localStorage.setItem("nombre", "Camila Ríos");
localStorage.setItem("edad", "27");

// Los valores siempre deben ser strings
```

getItem: recuperar datos

```
const nombre = localStorage.getItem("nombre");
console.log(nombre); // "Camila Ríos"

const edad = localStorage.getItem("edad");
```

```
console.log(edad); // "27" (string)

const inexistente = localStorage.getItem("noExiste");
console.log(inexistente); // null
```

removeItem: eliminar un dato

```
localStorage.removeItem("edad");
```

clear: limpiar todo

```
localStorage.clear(); // Elimina todo el localStorage del dominio
```

Guardando objetos en localStorage

—localStorage solo guarda strings. Para guardar objetos, usamos **JSON.stringify** y **JSON.parse**.

```
const usuario = {
  nombre: "Camila Ríos",
  email: "camila@aprendeya.com",
  edad: 27,
  cursos: ["HTML Básico", "JavaScript Básico"]
};

// Guardar objeto (convertir a JSON string)
localStorage.setItem("usuario", JSON.stringify(usuario));

// Recuperar objeto (convertir de JSON string)
const usuarioGuardado = JSON.parse(localStorage.getItem("usuario"));
console.log(usuarioGuardado.nombre); // "Camila Ríos"
console.log(usuarioGuardado.cursos[0]); // "HTML Básico"
```

Ejemplo práctico: progreso del curso

Camila implementó el seguimiento de progreso:

```
// Estado inicial
let progreso = {
  leccionesCompletadas: [],
  ultimoAcceso: null,
  cursoActual: null
};

// Cargar progreso guardado
function cargarProgreso() {
  const guardado = localStorage.getItem("progreso");
  if (guardado) {
```

```

    progreso = JSON.parse(guardado);
    console.log("Progreso cargado:", progreso);
  }
}

// Guardar progreso
function guardarProgreso() {
  progreso.ultimoAcceso = new Date().toISOString();
  localStorage.setItem("progreso", JSON.stringify(progreso));
  console.log("Progreso guardado");
}

// Completar una lección
function completarLeccion(idLeccion) {
  if (!progreso.leccionesCompletadas.includes(idLeccion)) {
    progreso.leccionesCompletadas.push(idLeccion);
    guardarProgreso();
    actualizarUI();
  }
}

// Actualizar la interfaz
function actualizarUI() {
  const contador = document.querySelector("#contador-lecciones");
  contador.textContent = `Lecciones completadas: ${progreso.leccionesCompletadas.length}`;
}

// Inicializar
cargarProgreso();
actualizarUI();

```

sessionStorage

—Hay un hermano de `localStorage` llamado **sessionStorage**. Funciona igual, pero los datos se borran cuando se cierra la pestaña.

```

// Guardar en sessionStorage (temporal)
sessionStorage.setItem("carrito", JSON.stringify(["Curso JS", "Curso HTML"]));

// sessionStorage se borra al cerrar la pestaña
// localStorage persiste hasta que se borre manualmente

```

Característica	localStorage	sessionStorage
Persistencia	Hasta borrado manual	Hasta cerrar pestaña
Capacidad	~5MB	~5MB
Compartido entre pestañas	Sí	No
Uso típico	Preferencias, progreso	Datos temporales, carrito

Cookies

—Las **cookies** son otra forma de almacenar datos, pero más antiguas y con más limitaciones. Se envían al servidor en cada petición.

```
// Crear una cookie (expira en 7 días)
document.cookie = "nombre=Camila; max-age=" + (7 * 24 * 60 * 60);

// Leer cookies
console.log(document.cookie);

// Eliminar cookie (max-age negativo)
document.cookie = "nombre=; max-age=0";
```

—Para la mayoría de casos, usa `localStorage` —dijo Diego—. Es más simple y no ralentiza las peticiones al servidor.

Aplicación en la plataforma

Camila integró localStorage en la plataforma:

```
// ===== MÓDULO DE ALMACENAMIENTO =====

const Almacenamiento = {
  // Guardar datos del usuario al registrarse
  guardarUsuario(datos) {
    const usuarios = this.obtenerUsuarios();
    datos.id = Date.now();
    usuarios.push(datos);
    localStorage.setItem("usuarios", JSON.stringify(usuarios));
    return datos;
  },

  // Obtener todos los usuarios
  obtenerUsuarios() {
    const datos = localStorage.getItem("usuarios");
    return datos ? JSON.parse(datos) : [];
  },

  // Guardar preferencias
  guardarPreferencias(prefs) {
    localStorage.setItem("preferencias", JSON.stringify(prefs));
  },

  // Obtener preferencias
  obtenerPreferencias() {
    const datos = localStorage.getItem("preferencias");
    return datos ? JSON.parse(datos) : { modoOscuro: false, idioma: "es" };
  },

  // Guardar progreso de un curso
```

```

guardarProgresoCurso(cursoId, leccionId, completado) {
  const progresos = this.obtenerProgresos();
  if (!progresos[cursoId]) progresos[cursoId] = {};
  progresos[cursoId][leccionId] = completado;
  localStorage.setItem("progresos", JSON.stringify(progresos));
},

// Obtener todos los progresos
obtenerProgresos() {
  const datos = localStorage.getItem("progresos");
  return datos ? JSON.parse(datos) : {};
},

// Calcular porcentaje de un curso
obtenerPorcentajeCurso(cursoId, totalLecciones) {
  const progresos = this.obtenerProgresos();
  const curso = progresos[cursoId];
  if (!curso) return 0;
  const completadas = Object.values(curso).filter(v => v).length;
  return Math.round((completadas / totalLecciones) * 100);
}
};

// ===== USO =====

// Al registrarse
formulario.addEventListener("submit", function(event) {
  event.preventDefault();
  const datos = {
    nombre: document.querySelector("#nombre").value,
    email: document.querySelector("#email").value
  };
  Almacenamiento.guardarUsuario(datos);
  console.log("Usuario registrado y guardado!");
});

// Al completar una lección
function marcarCompletada(leccionId) {
  Almacenamiento.guardarProgresoCurso("js-basico", leccionId, true);
  const porcentaje = Almacenamiento.obtenerPorcentajeCurso("js-basico", 15);
  document.querySelector("#progreso").style.width = porcentaje + "%";
  document.querySelector("#porcentaje-texto").textContent = `${porcentaje}%`;
}

```

Enigmas del Capítulo 8

Enigma 8.1: Contador de visitas

Crea un script que muestre cuántas veces has visitado una página. Usa `localStorage` para almacenar el contador. Cada vez que cargues la página, incrementa el contador y muéstralo.

Enigma 8.2: Tema persistente

Crea un botón que alterne entre modo claro y modo oscuro. Guarda la preferencia en localStorage. Al recargar la página, debe mantener la última selección.

Enigma 8.3: Lista de favoritos

Crea una página con una lista de cursos y un botón "Favorito" en cada uno. Al hacer clic, guarda el curso en una lista de favoritos en localStorage. Muestra la lista en una sección separada.

Enigma 8.4: Notas persistentes

Crea un textarea donde el usuario pueda escribir notas. Cada vez que se escriba (evento input), guarda el contenido en localStorage. Al recargar la página, el contenido debe aparecer automáticamente.

Lo que aprendiste

- **localStorage**: almacenamiento persistente en el navegador
 - **sessionStorage**: almacenamiento temporal (hasta cerrar pestaña)
 - Métodos: ``setItem``, ``getItem``, ``removeItem``, ``clear``
 - **JSON.stringify** y **JSON.parse**: convertir objetos a/desde strings
 - Las **cookies** existen pero son menos recomendadas para almacenamiento local
 - localStorage es ideal para preferencias, progreso y datos de sesión
-

Camila recargó la página. El progreso seguía ahí. Cerró el navegador, lo volvió a abrir, y el contador de lecciones completadas seguía intacto.

—Es como si la página tuviera memoria —dijo, maravillada.

—Eso es exactamente —respondió Diego—. Y mañana vamos a conectarla con el mundo exterior.

Capítulo 9: Conectando con el Mundo

Fetch y APIs

—Los cursos de la plataforma están hardcodeados —dijo Diego—. Están escritos directamente en el código. En una aplicación real, los cursos vendrían de un servidor.

—¿Cómo los traemos? —preguntó Camila.

—Con **fetch**. Es la forma moderna de hacer peticiones HTTP desde JavaScript.

¿Qué es una API?

—**API** significa **Application Programming Interface**. Es un intermediario que permite que dos aplicaciones se comuniquen.

En el contexto web, una **API REST** expone datos a través de URLs. Por ejemplo:

```
https://api.aprendeya.com/cursos
https://api.aprendeya.com/cursos/1
https://api.aprendeya.com/usuarios
```

—El backend expone estas URLs, y el frontend las consume con JavaScript.

Fetch: Peticiones GET

—La forma más común es obtener datos con una petición **GET**.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(function(respuesta) {
    return respuesta.json();
  })
  .then(function(datos) {
    console.log(datos);
  })
  .catch(function(error) {
    console.error("Error:", error);
  });
```

—`fetch` devuelve una **Promise**. Las promesas representan operaciones asíncronas:

operaciones que no se completan inmediatamente.

Promesas: then y catch

—JavaScript es **asíncrono**. Cuando pides datos a un servidor, no sabes cuánto tardará la respuesta. Las promesas manejan esta incertidumbre.

```
// Una promesa puede estar en 3 estados:
// pending (pendiente), fulfilled (resuelta), rejected (rechazada)

const promesa = fetch("https://api.aprendeya.com/cursos");

promesa
  .then(function(respuesta) {
    // La promesa se resolvió exitosamente
    if (!respuesta.ok) {
      throw new Error("Error HTTP: " + respuesta.status);
    }
    return respuesta.json(); // También devuelve una promesa
  })
  .then(function(cursos) {
    // Aquí tienes los datos listos para usar
    console.log("Cursos obtenidos:", cursos);
    mostrarCursos(cursos);
  })
  .catch(function(error) {
    // La promesa fue rechazada (error de red, servidor caído, etc.)
    console.error("Error al obtener cursos:", error);
    mostrarError("No se pudieron cargar los cursos");
  });
```

Encadenamiento de promesas

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(respuesta => respuesta.json())
  .then(usuario => {
    console.log("Usuario:", usuario.name);
    return fetch(`https://jsonplaceholder.typicode.com/posts?userId=${usuario.id}`);
  })
  .then(respuesta => respuesta.json())
  .then(posts => {
    console.log("Posts del usuario:", posts);
  })
  .catch(error => console.error("Error:", error));
```

Async / Await

—Si las promesas te parecen confusas —dijo Diego—, JavaScript moderno ofrece una sintaxis

más limpia: **async/await**.

```
// Función async: siempre devuelve una promesa
async function obtenerCursos() {
  try {
    const respuesta = await fetch("https://api.aprendeya.com/cursos");

    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    const cursos = await respuesta.json();
    console.log("Cursos:", cursos);
    return cursos;
  } catch (error) {
    console.error("Error:", error);
    return [];
  }
}

// Uso
obtenerCursos().then(cursos => {
  console.log(`Se obtuvieron ${cursos.length} cursos`);
});
```

—`await` pausa la ejecución hasta que la promesa se resuelva. Solo funciona dentro de funciones `async`.

Enfoque	Sintaxis	Ideal para
then/catch	<code>.then().catch()</code>	Cadenas cortas
async/await	<code>async function() { await }</code>	Flujo lineal, múltiples await

Manejo de errores

—Las peticiones a servidores pueden fallar por muchas razones. Siempre debes manejar los errores.

```
async function cargarCursos() {
  const contenedor = document.querySelector("#lista-cursos");
  const loader = document.querySelector("#loader");

  try {
    // Mostrar loader
    loader.style.display = "block";
    contenedor.innerHTML = "";

    const respuesta = await fetch("https://jsonplaceholder.typicode.com/posts");

    if (!respuesta.ok) {
      throw new Error(`Error del servidor: ${respuesta.status}`);
    }
  }
}
```

```

    }

    const cursos = await respuesta.json();
    mostrarCursos(cursos);

  } catch (error) {
    contenedor.innerHTML = `
      <div class="error">
        <p> No pudimos cargar los cursos.</p>
        <p>${error.message}</p>
        <button onclick="cargarCursos()">Reintentar</button>
      </div>
    `;
  } finally {
    loader.style.display = "none";
  }
}

function mostrarCursos(cursos) {
  const contenedor = document.querySelector("#lista-cursos");

  cursos.slice(0, 10).forEach(curso => {
    const tarjeta = document.createElement("div");
    tarjeta.classList.add("tarjeta-curso");
    tarjeta.innerHTML = `
      <h3>${curso.title}</h3>
      <p>${curso.body}</p>
    `;
    contenedor.appendChild(tarjeta);
  });
}

```

Mostrando datos en la UI

Camila conectó su plataforma con una API simulada:

```

// API pública de ejemplo (JSONPlaceholder)
const API_URL = "https://jsonplaceholder.typicode.com";

const AprendeYaAPI = {
  async obtenerCursos() {
    const res = await fetch(`${API_URL}/posts`);
    return res.json();
  },

  async obtenerCurso(id) {
    const res = await fetch(`${API_URL}/posts/${id}`);
    return res.json();
  },

  async obtenerComentarios(cursoId) {

```

```

    const res = await fetch(`${API_URL}/comments?postId=${cursoId}`);
    return res.json();
  }
};

// Cargar cursos y mostrarlos en tarjetas
async function inicializarPlataforma() {
  try {
    const cursos = await AprendeYaAPI.obtenerCursos();
    const contenedor = document.querySelector("#cursos-container");

    contenedor.innerHTML = cursos.slice(0, 8).map(curso => `
      <div class="curso-card" data-id="${curso.id}">
        <h3>${curso.title}</h3>
        <p>${curso.body.substring(0, 100)}...</p>
        <button class="btn-ver-curso">Ver curso</button>
      </div>
    `).join("");

    // Event delegation para los botones
    contenedor.addEventListener("click", async (event) => {
      if (event.target.classList.contains("btn-ver-curso")) {
        const card = event.target.closest(".curso-card");
        const id = card.dataset.id;
        await verDetalleCurso(id);
      }
    });

  } catch (error) {
    document.querySelector("#cursos-container").innerHTML = `
      <div class="alerta alerta-error">
        No se pudieron cargar los cursos.
        <button onclick="inicializarPlataforma()">Reintentar</button>
      </div>
    `;
  }
}

async function verDetalleCurso(id) {
  try {
    const curso = await AprendeYaAPI.obtenerCurso(id);
    const comentarios = await AprendeYaAPI.obtenerComentarios(id);

    const detalle = document.querySelector("#detalle-curso");
    detalle.innerHTML = `
      <h2>${curso.title}</h2>
      <p>${curso.body}</p>
      <h4>Comentarios (${comentarios.length})</h4>
      ${comentarios.map(c => `
        <div class="comentario">
          <strong>${c.name}</strong>
          <p>${c.body}</p>
        `)}
    `;
  }
}

```

```

    </div>
    `).join(""))}
    `;

    } catch (error) {
        console.error("Error al cargar detalle:", error);
    }
}

// Iniciar
inicializarPlataforma();

```

Enviando datos con POST

—No solo obtenemos datos. También podemos enviarlos.

```

async function registrarUsuario(nuevoUsuario) {
    try {
        const respuesta = await fetch("https://jsonplaceholder.typicode.com/users", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify(nuevoUsuario)
        });

        if (!respuesta.ok) {
            throw new Error("Error al registrar usuario");
        }

        const usuarioCreado = await respuesta.json();
        console.log("Usuario creado:", usuarioCreado);
        return usuarioCreado;

    } catch (error) {
        console.error("Error:", error);
        return null;
    }
}

// Uso
registrarUsuario({
    name: "Camila Ríos",
    email: "camila@aprendeya.com"
});

```

Enigmas del Capítulo 9

Enigma 9.1: Tu primera API

Usando ``https://jsonplaceholder.typicode.com/users``, obtén la lista de usuarios y muestra en consola solo los nombres y emails.

Enigma 9.2: Galería de fotos

Usando ``https://jsonplaceholder.typicode.com/photos``, obtén las primeras 10 fotos y muéstralas en una galería HTML. Cada foto debe mostrar la miniatura (`thumbnailUrl`) y el título.

Enigma 9.3: Carga con indicador

Modifica el enigma 9.2 para mostrar un mensaje "Cargando..." mientras se obtienen los datos, y reemplazarlo con la galería cuando lleguen los datos. Maneja el caso de error.

Enigma 9.4: Buscador de usuarios

Crea un input de búsqueda que filtre usuarios de la API ``https://jsonplaceholder.typicode.com/users`` en tiempo real. Muestra solo los usuarios cuyo nombre coincida con lo que el usuario escribe.

Lo que aprendiste

- **API**: interfaz para comunicar aplicaciones
 - **fetch**: API nativa para peticiones HTTP
 - **GET**: obtener datos del servidor
 - **POST**: enviar datos al servidor
 - **Promesas**: operaciones asíncronas con ``.then()`` y ``.catch()``
 - **async/await**: sintaxis más legible para código asíncrono
 - Manejo de **errores** con `try/catch`
 - Mostrar **loader** y estados de carga en la UI
-

—Ahora la plataforma ya no está aislada —dijo Camila, viendo cómo los cursos aparecían desde la API—. Se conecta con el mundo.

Diego asintió.

—Mañana es el gran día. La demo con los inversores. Vamos a integrar todo lo que has aprendido.

Camila miró su código. Variables, condicionales, funciones, arrays, objetos, DOM, eventos, formularios, localStorage, fetch. Había recorrido un largo camino en solo dos semanas.

Capítulo 10: La Gran Demo

Proyecto Final

Era el día de la demo. El sol entraba por las ventanas del piso 14, iluminando la sala de reuniones donde los inversores esperaban. Camila había dormido apenas cuatro horas. Pero su código estaba listo.

—¿Preparada? —preguntó Diego.

Camila asintió. Abrió su laptop y mostró la pantalla: la plataforma "AprendeYa" versión interactiva.

—Señores inversores —comenzó—, les presento el futuro de la educación online.

Proyecto: Mini Plataforma Educativa

Vamos a construir lo mismo que Camila presentó: una mini plataforma educativa con:

1. Login simulado
2. Catálogo de cursos dinámico
3. Seguimiento de progreso
4. Persistencia de datos

Estructura HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AprendeYa - Plataforma Educativa</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>
  <header>
    <div class="logo">AprendeYa</div>
    <nav id="nav-principal">
      <a href="#" data-seccion="cursos">Cursos</a>
      <a href="#" data-seccion="progreso">Mi Progreso</a>
      <a href="#" data-seccion="perfil">Perfil</a>
    </nav>
    <button id="btn-login" class="btn">Iniciar Sesión</button>
```

```

        <button id="btn-logout" class="btn oculto">Cerrar Sesión</button>
    </header>

    <main>
        <!-- Sección de Login -->
        <section id="seccion-login" class="seccion">
            <div class="login-card">
                <h2>Iniciar Sesión</h2>
                <form id="form-login">
                    <input type="text" id="login-nombre" placeholder="Nombre" required>
                    <input type="email" id="login-email" placeholder="Email" required>
                    <button type="submit" class="btn btn-primary">Entrar</button>
                </form>
                <p id="login-error" class="error-mensaje"></p>
            </div>
        </section>

        <!-- Sección de Cursos -->
        <section id="seccion-cursos" class="seccion">
            <h2>Cursos Disponibles</h2>
            <div id="loader" class="loader oculto">Cargando cursos...</div>
            <div id="cursos-container" class="grid-cursos"></div>
        </section>

        <!-- Sección de Progreso -->
        <section id="seccion-progreso" class="seccion oculto">
            <h2>Mi Progreso</h2>
            <div id="progreso-container"></div>
        </section>

        <!-- Sección de Perfil -->
        <section id="seccion-perfil" class="seccion oculto">
            <h2>Perfil de Usuario</h2>
            <div id="perfil-container"></div>
        </section>
    </main>

    <script src="app.js"></script>
</body>
</html>

```

JavaScript completo

```

// =====
// MÓDULO DE AUTENTICACIÓN
// =====
const Auth = {
    usuarioActual: null,

    init() {
        this.usuarioActual = JSON.parse(localStorage.getItem("usuarioActual"));
        if (this.usuarioActual) {

```

```

        UI.mostrarDashboard();
    }
    this.configurarEventos();
},

configurarEventos() {
    const form = document.querySelector("#form-login");
    form.addEventListener("submit", (e) => {
        e.preventDefault();
        this.iniciarSesion();
    });

    document.querySelector("#btn-logout").addEventListener("click", () => {
        this.cerrarSesion();
    });
},

iniciarSesion() {
    const nombre = document.querySelector("#login-nombre").value.trim();
    const email = document.querySelector("#login-email").value.trim();
    const errorDiv = document.querySelector("#login-error");

    if (!nombre || !email) {
        errorDiv.textContent = "Todos los campos son obligatorios";
        return;
    }

    if (!email.includes("@")) {
        errorDiv.textContent = "Email inválido";
        return;
    }

    this.usuarioActual = {
        id: Date.now(),
        nombre: nombre,
        email: email,
        fechaIngreso: new Date().toISOString()
    };

    localStorage.setItem("usuarioActual", JSON.stringify(this.usuarioActual));
    errorDiv.textContent = "";
    UI.mostrarDashboard();
},

cerrarSesion() {
    this.usuarioActual = null;
    localStorage.removeItem("usuarioActual");
    UI.mostrarLogin();
},

estaLogueado() {
    return this.usuarioActual !== null;
}

```

```

    }
  };

  // =====
  // MÓDULO DE CURSOS
  // =====
  const Cursos = {
    datos: [],
    API_URL: "https://jsonplaceholder.typicode.com/posts",

    async cargarCursos() {
      const loader = document.querySelector("#loader");
      const container = document.querySelector("#cursos-container");

      loader.classList.remove("oculto");
      container.innerHTML = "";

      try {
        const respuesta = await fetch(this.API_URL);

        if (!respuesta.ok) {
          throw new Error("Error al cargar cursos");
        }

        const datos = await respuesta.json();

        // Transformar datos a nuestro formato
        this.datos = datos.slice(0, 12).map((item, index) => ({
          id: index + 1,
          titulo: item.title,
          descripcion: item.body,
          lecciones: Math.floor(Math.random() * 20) + 5,
          nivel: ["Principiante", "Intermedio", "Avanzado"][Math.floor(Math.random() * 3)]
        }));

        this.renderizar(container);

      } catch (error) {
        container.innerHTML = `
          <div class="alerta-error">
            <p>No pudimos cargar los cursos.</p>
            <button onclick="Cursos.cargarCursos()" class="btn">Reintentar</button>
          </div>
        `;
      } finally {
        loader.classList.add("oculto");
      }
    },

    renderizar(container) {
      container.innerHTML = this.datos.map(curso => `
        <div class="curso-card" data-id="${curso.id}">

```

```

        <div class="curso-nivel nivel-`${curso.nivel.toLowerCase()}`">`${curso.nivel}</div>
        <h3>`${curso.titulo}</h3>
        <p>`${curso.descripcion.substring(0, 120)}...</p>
        <div class="curso-info">
            <span> `${curso.lecciones} lecciones</span>
        </div>
        <button class="btn btn-inscribirse ${Progreso.estaInscrito(curso.id) ? 'btn-completado' : ''}">
            `${Progreso.estaInscrito(curso.id) ? ' Inscrito' : 'Inscribirse'}`
        </button>
    </div>
`}).join("");

// Event delegation para botones de inscripción
container.addEventListener("click", (e) => {
    const btn = e.target.closest(".btn-inscribirse");
    if (btn) {
        const card = btn.closest(".curso-card");
        const cursoId = parseInt(card.dataset.id);
        this.toggleInscripcion(cursoId, btn);
    }
});
},

toggleInscripcion(cursoId, btn) {
    if (!Auth.estaLogueado()) {
        alert("Debes iniciar sesión para inscribirte");
        return;
    }

    if (Progreso.estaInscrito(cursoId)) {
        Progreso.desinscribir(cursoId);
        btn.textContent = "Inscribirse";
        btn.classList.remove("btn-completado");
    } else {
        Progreso.inscribir(cursoId);
        btn.textContent = " Inscrito";
        btn.classList.add("btn-completado");
    }
}
};

// =====
// MÓDULO DE PROGRESO
// =====
const Progreso = {
    datos: {},

    init() {
        const guardado = localStorage.getItem("progreso");
        this.datos = guardado ? JSON.parse(guardado) : {};
    },

```

```

inscribir(cursoId) {
  if (!this.datos[cursoId]) {
    this.datos[cursoId] = {
      inscrito: true,
      leccionesCompletadas: 0,
      fechaInscripcion: new Date().toISOString()
    };
    this.guardar();
  }
},

desinscribir(cursoId) {
  delete this.datos[cursoId];
  this.guardar();
},

estaInscrito(cursoId) {
  return this.datos[cursoId] && this.datos[cursoId].inscrito;
},

completarLeccion(cursoId) {
  if (!this.datos[cursoId]) return;
  this.datos[cursoId].leccionesCompletadas++;
  this.guardar();
},

obtenerProgreso(cursoId, totalLecciones) {
  if (!this.datos[cursoId]) return 0;
  return Math.round((this.datos[cursoId].leccionesCompletadas / totalLecciones) * 100);
},

guardar() {
  localStorage.setItem("progreso", JSON.stringify(this.datos));
},

renderizarProgreso() {
  const container = document.querySelector("#progreso-container");
  const cursosInscritos = Object.keys(this.datos);

  if (cursosInscritos.length === 0) {
    container.innerHTML = "<p>Aún no te has inscrito en ningún curso.</p>";
    return;
  }

  container.innerHTML = cursosInscritos.map(cursoId => {
    const curso = Cursos.datos.find(c => c.id === parseInt(cursoId));
    if (!curso) return "";
    const progreso = this.obtenerProgreso(cursoId, curso.lecciones);

    return `
      <div class="progreso-card">
        <h4>${curso.titulo}</h4>
    `;
  });
}

```

```

        <div class="barra-progreso">
            <div class="barra-llenado" style="width: ${progreso}%"></div>
        </div>
        <p>${progreso}% completado</p>
        <button class="btn btn-small" onclick="Progreso.completarLeccion(${cursoId}); Progreso.renderizar(
            +1 Lección
        </button>
    </div>
    `;
    }).join("");
}
};

// =====
// MÓDULO DE INTERFAZ
// =====
const UI = {
    mostrarLogin() {
        document.querySelector("#seccion-login").classList.remove("oculto");
        document.querySelector("#seccion-cursos").classList.add("oculto");
        document.querySelector("#seccion-progreso").classList.add("oculto");
        document.querySelector("#seccion-perfil").classList.add("oculto");
        document.querySelector("#btn-login").classList.remove("oculto");
        document.querySelector("#btn-logout").classList.add("oculto");
    },

    mostrarDashboard() {
        document.querySelector("#seccion-login").classList.add("oculto");
        document.querySelector("#seccion-cursos").classList.remove("oculto");
        document.querySelector("#seccion-progreso").classList.remove("oculto");
        document.querySelector("#seccion-perfil").classList.remove("oculto");
        document.querySelector("#btn-login").classList.add("oculto");
        document.querySelector("#btn-logout").classList.remove("oculto");

        this.mostrarBienvenida();
        this.configurarNavegacion();
        Cursos.cargarCursos();
        Progreso.renderizarProgreso();
    },

    mostrarBienvenida() {
        const perfilContainer = document.querySelector("#perfil-container");
        perfilContainer.innerHTML = `
            <div class="perfil-info">
                <p><strong>Nombre:</strong> ${Auth.usuarioActual.nombre}</p>
                <p><strong>Email:</strong> ${Auth.usuarioActual.email}</p>
                <p><strong>Miembro desde:</strong> ${new Date(Auth.usuarioActual.fechaIngreso).toLocaleDateString()}
            </div>
            `;
    },

    configurarNavegacion() {

```

```

document.querySelectorAll("[data-seccion]").forEach(enlace => {
  enlace.addEventListener("click", (e) => {
    e.preventDefault();
    const seccion = e.target.dataset.seccion;
    document.querySelectorAll("section.seccion").forEach(s => {
      if (s.id === `seccion-${seccion}`) {
        s.classList.remove("oculto");
      } else if (s.id !== "seccion-login") {
        s.classList.add("oculto");
      }
    });
    if (seccion === "progreso") Progreso.renderizarProgreso();
  });
});

// =====
// INICIALIZACIÓN
// =====
document.addEventListener("DOMContentLoaded", () => {
  Progreso.init();
  Auth.init();
});

```

La demo en acción

Camila hizo una demostración en vivo:

1. **Login:** Ingresó su nombre y email. La plataforma la reconoció.
2. **Cursos:** Los cursos aparecieron desde la API con un elegante loader.
3. **Inscripción:** Se inscribió en tres cursos. Los botones cambiaron a "Inscrito".
4. **Progreso:** Marcó lecciones completadas. Las barras de progreso se llenaron.
5. **Persistencia:** Cerró sesión, volvió a iniciar, y su progreso seguía intacto.

Los inversores asintieron, impresionados. La plataforma ya no era un prototipo estático. Era una aplicación interactiva.

—Esto es solo el comienzo —dijo Camila—. Lo que han visto hoy es JavaScript puro. Imagínense lo que podemos hacer con frameworks modernos.

Enigmas del Capítulo 10

Enigma 10.1: Mejora el diseño

Agrega estilos CSS para que la plataforma se vea profesional. Incluye: colores corporativos, animaciones suaves en las tarjetas, hover effects en botones, diseño responsive.

Enigma 10.2: Buscador de cursos

Agrega un input de búsqueda que filtre los cursos en tiempo real mientras el usuario escribe.

Enigma 10.3: Modo oscuro

Implementa un botón de modo oscuro que sea persistente (localStorage). Cambia los colores de toda la plataforma.

Enigma 10.4: Estadísticas de progreso

Agrega una sección de estadísticas que muestre: cursos totales, cursos inscritos, lecciones completadas, promedio de progreso. Calcula estos datos desde el módulo Progreso.

Lo que aprendiste en este libro

- **Variables:** ``const``, ``let``, tipos de datos
 - **Condicionales:** ``if/else``, ``switch``, ternario
 - **Funciones:** declaradas, expresadas, arrow functions
 - **Arrays y objetos:** métodos de iteración (``map``, ``filter``, ``find``)
 - **DOM:** seleccionar, modificar, crear elementos
 - **Eventos:** click, submit, input, mouseover
 - **Formularios:** validación, expresiones regulares
 - **localStorage:** persistencia de datos
 - **Fetch:** consumo de APIs, `async/await`
 - **Modularización:** separar el código en módulos
-

Los inversores aplaudieron. Marco, el CEO, sonreía de oreja a oreja.

—Camila —dijo uno de los inversores—, ¿cómo aprendiste todo esto en tres semanas?

Camila miró a Diego, que levantó su taza de café en un brindis silencioso.

—Con un buen mentor y muchas ganas de aprender —respondió.

Mientras guardaba su laptop, Camila sintió algo que no había sentido antes: confianza en sí misma como desarrolladora. No solo era diseñadora. Ahora también era alguien que podía darle vida a la web.

Y ese, pensó, era el verdadero poder de la interactividad.

Conclusión

El Viaje Continúa

Camila Ríos logró convertir una plataforma estática en una experiencia interactiva. Aprendió JavaScript desde cero en tres semanas, y demostró que con dedicación y las herramientas adecuadas, cualquiera puede aprender a programar.

Pero este no es el final. Es solo el comienzo.

¿Qué sigue?

JavaScript es un universo en expansión. Lo que has aprendido en este libro es la base sobre la que se construye todo lo demás.

Frameworks y librerías:

Tecnología	¿Para qué sirve?	¿Cuándo aprenderla?
React	Crear interfaces de usuario con componentes reutilizables	Después de dominar bien JavaScript y DOM
Vue	Framework progresivo para aplicaciones web	Alternativa más suave a React
Angular	Framework completo para aplicaciones empresariales	Cuando trabajes en proyectos grandes
Node.js	JavaScript en el servidor	Para ser desarrollador full-stack
Express	Framework para APIs con Node.js	Después de Node.js
TypeScript	JavaScript con tipos	Para proyectos grandes y en equipo

Conceptos para profundizar:

- **ES6+:** Las características modernas de JavaScript
- **Programación asíncrona:** Promesas, async/await a fondo
- **Módulos:** Import y export
- **Testing:** Jest, Vitest
- **Bundlers:** Vite, Webpack
- **Bases de datos:** Firebase, MongoDB
- **Despliegue:** Netlify, Vercel, GitHub Pages

Proyectos para practicar

1. **App del clima:** Usa una API del clima gratuita
2. **Todo List:** La app clásica para practicar CRUD
3. **Juego de memoria:** HTML, CSS y JavaScript puro
4. **Blog personal:** Con artículos desde una API

5. E-commerce básico: Carrito de compras con localStorage

Recursos recomendados

- **MDN Web Docs:** La documentación oficial de JavaScript
- **JavaScript.info:** Tutorial completo y actualizado
- **freeCodeCamp:** Cursos gratuitos con certificación
- **Codecademy:** Plataforma interactiva para practicar
- **YouTube:** Canales como Fazt, midulive, HolaMundo

Palabras finales

Aprender a programar es como aprender un nuevo idioma. Al principio todo es confuso, las palabras no tienen sentido, y la sintaxis parece arbitraria. Pero con la práctica, el código comienza a fluir. Empiezas a pensar en términos de variables, funciones y eventos. Ves un problema y automáticamente empiezas a diseñar una solución en tu mente.

JavaScript te ha dado el poder de la interactividad. Úsalo para crear, para resolver problemas, para hacer del mundo digital un lugar mejor.

Como dijo Camila al final de su demo: "Esto es solo el comienzo".

El viaje continúa. Sigue aprendiendo, sigue creando, sigue compartiendo.

— *Alex Goyzueta Delgado*

Lima, 2026

Apéndice: Soluciones a los Enigmas

Capítulo 1: El Primer Script

Enigma 1.1: Tu primer programa

```
const miNombre = "Tu nombre aquí";
let miEdad = 25;
const lenguajeFavorito = "JavaScript";

console.log(`Hola, me llamo ${miNombre}, tengo ${miEdad} años y mi lenguaje favorito es ${lenguajeFavorito}`);
```

Enigma 1.2: Detecta el tipo

```
typeof "42"    // "string"
typeof 42      // "number"
typeof true    // "boolean"
typeof null    // "object" (error histórico de JavaScript)
typeof undefined // "undefined"
typeof "true"  // "string"
```

El caso `typeof null === "object"` es un error conocido de JavaScript que no se corrigió por compatibilidad histórica.

Enigma 1.3: El mensaje de la plataforma

```
const curso = "JavaScript Básico";
let progreso = 15;
console.log(`Estás tomando el curso ${curso}. Tu progreso es ${progreso}%.`);
```

Enigma 1.4: Conversión de tipos

```
console.log("5" + 3); // "53" (concatenación)
console.log("5" - 3); // 2 (conversión a número)
console.log("5" * 3); // 15 (ambos se convierten a números)
console.log("Hola" + 5); // "Hola5" (concatenación)
```

Capítulo 2: Tomando Decisiones

Enigma 2.1: Calculadora de notas

```

let nota = 85;

if (nota >= 90) {
  console.log("Excelente");
} else if (nota >= 70) {
  console.log("Bien");
} else if (nota >= 50) {
  console.log("Regular");
} else {
  console.log("Necesitas mejorar");
}

```

Enigma 2.2: Acceso a la plataforma

```

let logueado = true;
let tieneSuscripcion = false;
let esAdmin = true;

if (logueado && (tieneSuscripcion || esAdmin)) {
  console.log("Acceso concedido");
} else if (logueado && !tieneSuscripcion && !esAdmin) {
  console.log("Debes adquirir una suscripción");
} else {
  console.log("Por favor, inicia sesión");
}

```

Enigma 2.3: Día laboral o finde

```

const dia = "sabado";

switch (dia) {
  case "lunes":
  case "martes":
  case "miercoles":
  case "jueves":
  case "viernes":
    console.log("Día laboral");
    break;
  case "sabado":
  case "domingo":
    console.log("Fin de semana");
    break;
  default:
    console.log("Día no válido");
}

```

Enigma 2.4: ¿Par o impar?

```

function esPar(numero) {
  return numero % 2 === 0;
}

```

```
console.log(`15: ${esPar(15) ? "par" : "impar"}`);
console.log(`22: ${esPar(22) ? "par" : "impar"}`);
console.log(`7: ${esPar(7) ? "par" : "impar"}`);
console.log(`100: ${esPar(100) ? "par" : "impar"}`);
```

Capítulo 3: Las Funciones del Conocimiento

Enigma 3.1: Tu primera función

```
function esMayorDeEdad(edad) {
  return edad >= 18;
}

console.log(esMayorDeEdad(15)); // false
console.log(esMayorDeEdad(18)); // true
console.log(esMayorDeEdad(21)); // true
console.log(esMayorDeEdad(0));  // false
```

Enigma 3.2: Calculadora básica

```
const sumar = (a, b) => a + b;
const restar = (a, b) => a - b;
const multiplicar = (a, b) => a * b;

function calculadora(operacion, a, b) {
  if (operacion === "suma") return sumar(a, b);
  if (operacion === "resta") return restar(a, b);
  if (operacion === "multiplica") return multiplicar(a, b);
  return "Operación no válida";
}

console.log(calculadora("suma", 10, 5));      // 15
console.log(calculadora("resta", 10, 5));     // 5
console.log(calculadora("multiplica", 10, 5)); // 50
```

Enigma 3.3: Validación de email

```
function validarEmail(email) {
  return email.includes("@") && email.includes(".", email.indexOf("@"));
}

console.log(validarEmail("camila@email.com")); // true
console.log(validarEmail("invalido@"));       // false
console.log(validarEmail("sinarroba.com"));   // false
```

Enigma 3.4: Conversión de temperatura

```
const celsiusAFahrenheit = celsius => celsius * 9/5 + 32;

console.log(celsiusAFahrenheit(0)); // 32
```

```
console.log(celsiusAFahrenheit(100)); // 212
console.log(celsiusAFahrenheit(37)); // 98.6
```

Capítulo 4: Listas y Colecciones

Enigma 4.1: Tu lista de compras

```
let listaCompras = ["pan", "leche", "huevos", "arroz", "frijoles"];
listaCompras.push("azúcar");
listaCompras.shift();
console.log(listaCompras);
console.log(`Longitud: ${listaCompras.length}`);
```

Enigma 4.2: Catálogo de cursos

```
const cursos = [
  { titulo: "HTML Básico", duracion: "5h", precio: 0 },
  { titulo: "CSS Avanzado", duracion: "8h", precio: 29.99 },
  { titulo: "JavaScript", duracion: "10h", precio: 0 }
];

const titulos = cursos.map(curso => curso.titulo);
console.log(titulos);

const cursosGratis = cursos.filter(curso => curso.precio === 0);
console.log(cursosGratis);
```

Enigma 4.3: Objeto estudiante

```
const estudiante = {
  nombre: "Camila Ríos",
  edad: 27,
  cursos: ["HTML", "CSS", "JavaScript"],
  direccion: {
    ciudad: "Lima",
    pais: "Perú"
  },
  presentarse() {
    return `Hola, soy ${this.nombre} de ${this.direccion.ciudad}, ${this.direccion.pais}.`;
  }
};

console.log(estudiante.presentarse());
```

Enigma 4.4: Transformando datos

```
const nums = [5, 12, 8, 21, 3, 15];

const mayores = nums.filter(n => n > 10);
console.log(mayores); // [12, 21, 15]
```

```
const dobles = nums.map(n => n * 2);
console.log(dobles); // [10, 24, 16, 42, 6, 30]

const primeroMayor = nums.find(n => n > 10);
console.log(primeroMayor); // 12
```

Capítulo 5: La Vida en la Página

Enigma 5.1: Botón que cambia de color

```
const boton = document.querySelector("#miBoton");

boton.addEventListener("click", function() {
  const color = "#" + Math.floor(Math.random() * 16777215).toString(16);
  this.style.backgroundColor = color;
});
```

Enigma 5.2: Contador de clics

```
let contador = 0;
const btnContador = document.querySelector("#btn-contador");
const display = document.querySelector("#display");

btnContador.addEventListener("click", () => {
  contador++;
  if (contador > 10) contador = 0;
  display.textContent = contador;
});
```

Enigma 5.3: Mouseover en tarjetas

```
document.querySelectorAll(".tarjeta").forEach(tarjeta => {
  tarjeta.addEventListener("mouseover", function() {
    this.style.border = "2px solid green";
  });
  tarjeta.addEventListener("mouseout", function() {
    this.style.border = "1px solid #ccc";
  });
});
```

Enigma 5.4: Input en vivo

```
const input = document.querySelector("#miInput");
const parrafo = document.querySelector("#vista-previa");

input.addEventListener("input", function() {
  parrafo.textContent = this.value || "Escribe algo...";
});
```

Capítulo 6: Transformando la Interfaz

Enigma 6.1: Lista de tareas

```
const input = document.querySelector("#nueva-tarea");
const btn = document.querySelector("#agregar");
const lista = document.querySelector("#lista-tareas");

btn.addEventListener("click", () => {
  const texto = input.value.trim();
  if (texto === "") return alert("Escribe una tarea");

  const li = document.createElement("li");
  li.textContent = texto;
  lista.appendChild(li);
  input.value = "";
});
```

Enigma 6.2: Tarjetas de colores

```
const btnAgregar = document.querySelector("#agregar-tarjeta");
const contenedor = document.querySelector("#contenedor-tarjetas");

btnAgregar.addEventListener("click", () => {
  const tarjeta = document.createElement("div");
  tarjeta.className = "tarjeta-color";
  tarjeta.style.backgroundColor = "#" + Math.floor(Math.random() * 16777215).toString(16);
  tarjeta.innerHTML = '<button class="eliminar">X</button>';
  contenedor.appendChild(tarjeta);
});

contenedor.addEventListener("click", (e) => {
  if (e.target.classList.contains("eliminar")) {
    e.target.parentElement.remove();
  }
});
```

Enigma 6.3: Modo oscuro

```
const btnModo = document.querySelector("#btn-modo");
const body = document.body;

btnModo.addEventListener("click", () => {
  body.classList.toggle("modo-oscuro");
  btnModo.textContent = body.classList.contains("modo-oscuro") ? "Modo Claro" : "Modo Oscuro";
});
```

Enigma 6.4: Galería dinámica

```
const imagenes = [
  "https://via.placeholder.com/150/ff0000",
  "https://via.placeholder.com/150/00ff00",
```

```

    "https://via.placeholder.com/150/0000ff"
  ];

  const galeria = document.querySelector("#galeria");

  imagenes.forEach(url => {
    const img = document.createElement("img");
    img.src = url;
    img.addEventListener("mouseover", () => img.style.border = "3px solid gold");
    img.addEventListener("mouseout", () => img.style.border = "none");
    galeria.appendChild(img);
  });

```

Capítulo 7: Formularios Inteligentes

Enigma 7.1: Validación de edad

```

function validarEdad(edad) {
  const valor = parseInt(edad);
  if (isNaN(valor) || valor < 18 || valor > 120) {
    return "La edad debe ser un número entre 18 y 120";
  }
  return "";
}

```

Enigma 7.2: Confirmación de contraseña

```

function validarConfirmacion(password, confirmacion) {
  if (password !== confirmacion) {
    return "Las contraseñas no coinciden";
  }
  return "";
}

```

Enigma 7.3: Contador de caracteres

```

const textarea = document.querySelector("#biografia");
const contador = document.querySelector("#contador");
const max = 200;

textarea.addEventListener("input", function() {
  const restantes = max - this.value.length;
  contador.textContent = `${restantes} caracteres restantes`;
  contador.style.color = restantes < 0 ? "red" : "#666";
});

```

Enigma 7.4: Validación de nombre de usuario

```

function validarNombreUsuario(nombre) {
  const regex = /^[a-z][a-z0-9_]{3,14}$/;

```

```
return regex.test(nombre);
}
```

Capítulo 8: Guardando el Progreso

Enigma 8.1: Contador de visitas

```
let visitas = localStorage.getItem("contadorVisitas") || 0;
visitas = parseInt(visitas) + 1;
localStorage.setItem("contadorVisitas", visitas);
console.log(`Has visitado esta página ${visitas} veces`);
```

Enigma 8.2: Tema persistente

```
const temaGuardado = localStorage.getItem("tema");
if (temaGuardado === "oscuro") document.body.classList.add("modo-oscuro");

document.querySelector("#btn-tema").addEventListener("click", () => {
  document.body.classList.toggle("modo-oscuro");
  const tema = document.body.classList.contains("modo-oscuro") ? "oscuro" : "claro";
  localStorage.setItem("tema", tema);
});
```

Enigma 8.3: Lista de favoritos

```
function toggleFavorito(cursoId) {
  let favoritos = JSON.parse(localStorage.getItem("favoritos") || "[]");
  const index = favoritos.indexOf(cursoId);
  if (index === -1) favoritos.push(cursoId);
  else favoritos.splice(index, 1);
  localStorage.setItem("favoritos", JSON.stringify(favoritos));
  mostrarFavoritos();
}
```

Enigma 8.4: Notas persistentes

```
const notas = document.querySelector("#notas");
notas.value = localStorage.getItem("notas") || "";

notas.addEventListener("input", function() {
  localStorage.setItem("notas", this.value);
});
```

Capítulo 9: Conectando con el Mundo

Enigma 9.1: Tu primera API

```

fetch("https://jsonplaceholder.typicode.com/users")
  .then(res => res.json())
  .then(usuarios => {
    usuarios.forEach(u => console.log(`${u.name}: ${u.email}`));
  });

```

Enigma 9.2: Galería de fotos

```

async function cargarGaleria() {
  const res = await fetch("https://jsonplaceholder.typicode.com/photos");
  const fotos = await res.json();
  const galeria = document.querySelector("#galeria");

  fotos.slice(0, 10).forEach(foto => {
    const div = document.createElement("div");
    div.innerHTML = `<p>${foto.title}</p>`;
    galeria.appendChild(div);
  });
}

```

Enigma 9.3: Carga con indicador

```

async function cargarGaleria() {
  const loader = document.querySelector("#loader");
  const galeria = document.querySelector("#galeria");

  loader.style.display = "block";

  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/photos");
    if (!res.ok) throw new Error("Error");
    const fotos = await res.json();
    galeria.innerHTML = fotos.slice(0, 10).map(f =>
      `<div><p>${f.title}</p></div>`
    ).join("");
  } catch {
    galeria.innerHTML = "<p>Error al cargar imágenes</p>";
  } finally {
    loader.style.display = "none";
  }
}

```

Enigma 9.4: Buscador de usuarios

```

let usuarios = [];

async function init() {
  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  usuarios = await res.json();
}

document.querySelector("#buscador").addEventListener("input", function() {

```

```
const filtro = this.value.toLowerCase();  
const resultados = usuarios.filter(u => u.name.toLowerCase().includes(filtro));  
console.log(resultados);  
});
```

Capítulo 10: La Gran Demo

Las soluciones del capítulo 10 son proyectos abiertos. No hay una única respuesta correcta. Evalúa tu solución según:

- **Enigma 10.1:** ¿La plataforma se ve profesional y responsiva?
- **Enigma 10.2:** ¿El buscador filtra correctamente en tiempo real?
- **Enigma 10.3:** ¿El modo oscuro persiste al recargar la página?
- **Enigma 10.4:** ¿Las estadísticas se calculan correctamente desde los datos reales?

Sobre el Autor

Alex Goyzueta Delgado

Analista de datos, escritor y divulgador de tecnología. Peruano, nacido en Ancon Lima.

Alex combina su formación en análisis de datos con una profunda pasión por la narrativa y la educación. Cree firmemente que la mejor forma de aprender herramientas técnicas es a través de historias que conecten con nuestras emociones y experiencias cotidianas.

Trilogía de Excel:

- **Excel Básico: El Legado de las Fórmulas** — Aprende Excel desde cero con Sofía Mendoza

- **Excel Intermedio: Los Secretos del Taller** — Funciones lógicas, BUSCARV, tablas dinámicas

- **Excel Avanzado: El Imperio de los Datos** — Macros, Power Query, dashboards profesionales

Serie de Programación:

- **JavaScript Básico: El Poder de la Interactividad** — Aprende JavaScript desde cero con Camila Ríos

Contacto:

- Correo: alexgoyzueta2018@gmail.com
- Temas: Excel, JavaScript, análisis de datos, narrativa educativa, productividad, SQL, Python
- Repositorio: github.com/SistGoyPeru/Libros

"El conocimiento no se crea ni se destruye: se transforma. De madera a muebles. De números a historias. De código a interactividad."